



Internet protocol services

(February 10, 2011)

Copyright by SERCOS International 2011 - All rights reserved.

Version 1.3-1.2

Developed by the SERCOS III Working Group "TWG NRT-Communication".

SERCOS International e.V. (SI) with its head offices in Süssen, Germany (hereafter referred to as "Licensor") consents to the use of specifications only under the following terms and conditions. The license granted with this certificate can exclusively be acquired from SI.

1. Specifications

1.1 The object of this licensing contract is the technical specification issued by the licensor, in particular but not restricted to the printed and electronic version (hereafter referred to as "SPECIFICATION").

1.2 The SPECIFICATION provided herewith was developed by the licensor's members or other staff for the licensor. For clarification it is emphasized that the SPECIFICATION is not an industrial standard recognized by a standardization body or in any other way.

2. Rights and duties of the licensor

2.1 The licensor herewith grants the licensee the right to use the SPECIFICATION exclusively for the purpose of developing and promoting products according to the SPECIFICATION. The licensee may copy the SPECIFICATION for this purpose and for data back-up purposes.

2.2 The licensee is not entitled to modify or extract individual parts of the SPECIFICATION, unless expressly permitted by law. The licensee is also not entitled to remove alphanumerical

labeling, markings or copyright notes from the SPECIFICATION, and if the licensee is entitled to reproduce copies of the SPECIFICATION this must be copied in its original state.

2.3 The licensee is not entitled to publish, market or sell the SPECIFICATION. The licensee is however entitled to transfer the rights of use of a SPECIFICATION to a third party under the condition that the licensee concludes a written contract with the third party which corresponds to all the terms and conditions contained in this paragraph

3. Licensor's liability

3.1 The licensor is not obligated to improve or modify the SPECIFICATION and he accepts no liability if the SPECIFICATION or future versions of it are not approved as industry standard. 3.2 The SPECIFICATION is supplied free to all the licensor's members and "without warranty." All liability and warranty for the SPECIFICATION – irrespective of legal basis – e.g. regarding quality or title, correctness, free of defects, lack of third party claims or in relation to completeness and/or suitability for a certain purpose are excluded.

The licensor is not liable for damages which result from the use of the SPECIFICATION, or for damages which are caused by the design of the hardware or software according to the information made available in the SPECIFICATION. All liability for consequential damages, special damages or indirect damages, including but not restricted to loss of profits, loss of use and interruptions to business are excluded.

3.3 The afore-mentioned disclaimer does not apply if a compulsory statutory liability exists e.g., according to product liability laws, in the case of intent, gross negligence, death, physical injury or damage to health or intentional concealment of defects or with a breach of significant contractual obligations. Liability for damages which have resulted as a breach of significant contractual obligations are restricted however to foreseeable damages which are usually covered by insurance, except in cases of intent or gross negligence. The existing condition does not lead to a reversal of the burden of proof to the detriment of the licensee.

4. Venue of jurisdiction and applicable law

The sole venue of jurisdiction is the main head offices of the licensor.

4.2 All relationships resulting from this contract are subject to the German material law, whereas the United Nations Convention on Contracts for the International Sale of Goods (CISG) does not apply and is excluded.

SERCOS International e.V.
Kueblerstrasse 1
73079 Suessen, Germany
Phone: +49(0)7162-9468-65
Fax: +49(0)7162-9468-66
E-Mail: info@sercos.de

1 Glossary.....	5
2 Introduction	7
3 General S/IP definitions	8
3.1 Version	8
3.2 Data types.....	8
3.3 Alignment.....	8
3.4 Services	8
3.5 Node Identifier	9
4 Transport Protocols	9
4.1 UDP based S/IP Protocol Handling	9
4.1.1 Limitations of PDU size	11
4.1.2 Reliability of Transmission	11
4.1.3 Unicast Services	11
4.1.4 Broadcast Services	11
4.2 TCP based S/IP Protocol Handling	12
4.2.1 S/IP Connections	12
4.3 TFTP.....	15
5 Communication	15
5.1 S/IP Header	17
5.1.1 Transaction ID.....	17
5.1.2 Message Types	18
5.2 Exception handling.....	19
5.2.1 Exception structure.....	20
5.2.2 Common Error Codes.....	20
6 Services	21
6.1 S/IP Connection Services	21
6.1.1 Connect Service.....	21
6.1.2 Ping Service.....	22
6.2 Explore and IP Configuration Services.....	23
6.2.1 Browse Service.....	23
6.2.2 SetIp Service	25
6.2.3 Interfaces Service	27
6.2.4 Slaves Service	28
6.2.5 BrowseOnSlaveInterface Service.....	29
6.3 Identification Services.....	30
6.3.1 Identify Service	30
6.3.2 Nameplate Service.....	31
6.3.3 NameplateBroadcast Service.....	32
6.4 SERCOS Parameter Access	33
6.4.1 Parameter data	33
6.4.2 IDNs	34
6.4.3 Parameter Service Specific Error Codes	34
6.4.4 ReadEverything Service	34
6.4.5 ReadOnlyData Service	36
6.4.6 ReadDescription Service	37
6.4.7 ReadDataStatus Service	38
6.4.8 ReadSegment Service.....	39
6.4.9 WriteName Service	40
6.4.10 WriteAttribute Service	40

6.4.11 WriteUnit Service.....	41
6.4.12 WriteMinMax Service.....	41
6.4.13 WriteData Service	42
6.4.14 WriteDataBits Service.....	43
6.5 Device Management Services	43
6.5.1 Reset Service	43
6.5.2 TFTP Firmware Management / Update	45
6.6 Other Services	45
6.6.1 SupportedUDPServices Service.....	45
6.6.2 Watchdog Service	47
7 Classification.....	49
8 Appendix A	51
8.1 Local Browsing	51
8.2 Remote Browsing.....	53

1 Glossary

AT	acknowledge telegram
attribute	attribute (of a parameter): • defines type and length of a parameter • element 3
connection	Connections: • are parts of the real-time data field of AT and MDT • consisting of connection control and the selected parameters • have one producer and at least one consumer • are unidirectional • enable direct data transfer: - from one slave to other slaves - from one slave to the master and other slaves - from the master to other slaves Both the master and the slaves may support several connections.
device	synonym for SERCOS III device
drive	synonym for power drive system
element	synonym for data block element
Ethernet	Ethernet, compatible to ISO/IEC 8802-3 standard
FSP	• function specific profile, according to SERCOS III • FAL service protocol, according to ISA-50.02, Part 6-1998
identification number	[abbr.: IDN] • designation of a parameter (and its appropriate data block elements 2 to 6) • data block element 1 There are two types of an IDN: • 32 bit significant (to denote: S-0-nnnn.SI.SE) • 16 bit significant (to denote: S-0-nnnn ; SI = SE = 0 and hence not denoted)
IDN	identification number (of a parameter)
INFO	service channel container for transmission of elements
IP	Internet protocol
little endian	describes a model of memory organisation which stores the least significant octet at the lowest address, or for transfer, which transfers the lowest order octet first
MAC	media access control
master	SERCOS III device that establishes cyclic communication of other SERCOS III devices (i.e., slaves)
MS	bidirectional data transfer between master and one slave (point-to-point) using real-time telegrams

name	name (of a parameter): • element 2 • octet string according to UTF8
node	according to OSI Reference Model: • single DL entity as it appears on one local link • a unit in an IP Network with at least one network interface
NRT	non real-time, the opposite of real-time
operation mode	• examples for drive operation mode are: position control, velocity control, positioning, etc. • operation mode might be parametrized in PL and executed in OL
parameter	• SERCOS III specific data format • element 7
PDU	protocol data unit
port	cable connection on a node
protocol	convention about the data formats, time sequences and error correction in the data exchange of communication systems
SERCOS	• abbr. for serial real-time communication system • synonym for SERCOS 2 • synonym for SERCOS III
SERCOS address	[abbr.: ADR] freely selectable address of a slave in the range of $(1 \leq \text{ADR} \leq 511)$ which has to be unique in the SERCOS III network (ADR = 0) might occur only during CP0, CP1 and CP2
SERCOS III	[abbr. for: third generation SERCOS] an industrial standard which specifies real-time Ethernet communication for automation, i.e.,: • SERCOS III network, • SERCOS III devices and • parameter
SERCOS III network	• is a real-time Ethernet specified by SERCOS III • consists of at least two SERCOS III devices which are able to run cyclic communication among themselves (i.e., one master and at least one slave) • might contain additional nodes as long as they do not interfere with the cyclic communication (insofar no switches allowed)
service channel	[abbr.: SVC] non real-time transmission inside of real-time telegrams upon master request; Only parameters may be sent.
SI	structure instance
slave	SERCOS III device whose cyclic communication will be established by another SERCOS III device (i.e., by the master)
SVC	service channel
TFTP	Trivial File Transfer Protocol (TFTP) is a file transfer protocol, with the

	functionality of a very basic form of File Transfer Protocol see RFC 1350
topology	physical network architecture with respect to the connection between the nodes of the network
TWG	technical working group, a SERCOS board
unit	unit (of a parameter): • element 4 • octet string according to UTF8

2 Introduction

The Internet protocol services specify different services for NRT communication participants in a SERCOS III network.

They use the following protocols:

- UDP
- TCP
- TFTP

Note: Internet protocol services assume a well configured IP network

The services are categorized according their functionality as follows:

- **S/IP Connection Services:** Connection services, which are required for TCP based S/IP data exchange
- **Explore & IP Configuration Services:** Services for:
 - o Detection of devices in SERCOS III networks
 - o IP configuration of devices to enable point to point communication
- **Identification Services:** Services for the identification of S/IP devices
- **SERCOS Parameter Access:** Read and write access to SERCOS III parameters
- **Device Management Services:**
 - o Restart devices
 - o TFTP Firmware Management
- **Other Services:**
 - o Identification of supported services
 - o Supervision of services based on a watchdog mechanism

By the help of these services, the following use cases become realizable:

- Identification and configuration of IP devices within SERCOS III networks
- Desk test of a device
- Maintenance or diagnosis of devices within an industrial plant

3 General S/IP definitions

3.1 Version

The version of S/IP protocol described in this specification is 0x01.

3.2 Data types

The following data types are used within the S/IP PDUs.

Table 1: Definition of Data types

Type	Definition	Comment
bool	1 octet	coding: 0 --> false, != 0 --> true
byte, uint8	1 octet unsigned integer	
int8	1 octet signed integers	
int16	2 octet signed integers	coded in little endian.
int32	4 octet signed integers	coded in little endian.
uint16	2 octet unsigned integers	coded in little endian.
uint32	4 octet unsigned integers	coded in little endian.
string	utf-8 coded	Fix-Len: There are fixed number of reserved bytes. The unused bytes shall be zero. There is no terminating 0, if the coding completely fills up the reserved number of bytes! Variable-Len: A leading explicit length contains the number of bytes. There is no terminating 0.

The SERCOS IDN data (min, max, operation data) is exchanged as byte arrays (see Parameter data).

3.3 Alignment

There are no spare octets due to an alignment.

3.4 Services

S/IP offers different services, which can be accessed using different transport protocols. Due to specific limitations of the used protocols, not every transport protocol can be used for each service.

3.5 Node Identifier

Since the nodes in a SERCOS III network may have more than one network interface, a node may be detected multiple times. In order to identify a node in network uniquely, another identifier, the so called *node identifier*, is introduced. The node identifier consists of six bytes. For slave devices the value of the node identifier shall be equal to the value of S-0-1019 MAC Address.

For nodes that support S/IP but do not have any SERCOS III slave interface, the MAC address of the first Ethernet adapter shall be used as node identifier. Nodes without Ethernet interface are not supported by S/IP.

4 Transport Protocols

Internet protocol services use TCP, UDP and TFTP as transport protocols for its services. Depending on the service it can be transmitted via one or more of these transport protocols. These protocols run on top of the internet protocol IP. S/IP doesn't support any fragmentation on the IP layer.

The following table gives an overview of the differences between TCP, UDP and TFTP based S/IP transmission.

Restriction/Addition	TCP	UDP	TFTP
Data exchange	Based on TCP sockets	Based on UDP datagrams	Based on UDP datagrams
Limitation of data size	No limitation	Depends on MTU size	TFTP does not support the transfer of files which are larger than 32MB, required minimum MTU size is 576 Byte
Reliable transmission	yes	no	yes
Flow control	yes	no	yes
Multicast communication	no	yes	no
Requires additional S/IP connections	yes	no	no

4.1 UDP based S/IP Protocol Handling

UDP based S/IP requests are generally transmitted to UDP server port 35021 (0x88cd). The client shall use a socket bound to any port except 35021.

Depending on the used service S/IP uses broadcast or unicast UDP telegrams. In general unicasts are used to address single S/IP nodes and broadcasts are used to address multiple nodes.

4.1.1 Limitations of PDU size

The maximum size of UDP based S/IP transmission is restricted to the configured MTU size.

The following restrictions to the response packet can be derived by considering a regular UDP packet. Every S/IP telegram contains a fixed part which is 50 octets in size. i.e.

- 14 octets Ethernet II Header
- 20 octets IP header without optional fields
- 8 octets UDP header
- 8 octets S/IP header (Transaction ID, Message Type)

Hence minimum size of response PDU is 50 octets. If the total size of a S/IP response exceeds the configured MTU size, the server must respond by sending an exception (see Exception handling).

Note: In order to be able to use TFTP over UDP the MTU size has to be at least 576 octets, see also RFC-879.

4.1.2 Reliability of Transmission

In general UDP does not guarantee the exchange of messages. Request or response messages may be lost during the communication.

In order to overcome this problem, the client shall implement a timeout for each request. After the timeout, the client may send the same request again with the same transaction ID or new transaction ID.

The server will process each incoming request and send the response back to the client. The server shall not implement a functionality to ensure that duplicated requests will be executed only once.

4.1.3 Unicast Services

S/IP services using UDP unicasts can be used if server and client are in the same or in different subnets. The response of the server response is a UDP unicast too.

4.1.4 Broadcast Services

S/IP uses UDP broadcasts to

- reach many devices with one request
- enable communication to non well configured devices.
E. g. a device may be connected to the same physical network, but has an unknown or invalid IP configuration. The IP address may be of an invalid or different subnet.

UDP broadcast telegrams can be responded by the server with UDP unicast or UDP broadcast messages. If the IP subnet of the client is different from the IP subnet of the server, the server shall send the response as a broadcast message. Otherwise the response shall be sent as a unicast message to avoid CPU load on other devices.

4.2 TCP based S/IP Protocol Handling

S/IP requests are generally transmitted to TCP server port 35021 (0x88cd). The client can use any port assigned by its TCP stack.

If TCP based S/IP is used, an S/IP connection has to be established before further S/IP services can be requested.

4.2.1 S/IP Connections

The S/IP connection provides the following features:

- The S/IP connection is established by the client.
- The S/IP connection is timeout controlled by the client and the server (see Timeouts).
- The S/IP connection properties including timeouts, version information and supported message types are negotiated during connection start-up.
- The server may offer several connections. The client may connect to one server several times by using several TCP sockets.
- While shutting down the connection, all the resources used internally should be released in the client and the server.

4.2.1.1 TCP based communication initialization

The S/IP connection has to be established after the initiation of the TCP socket connection.

The services required for the connection initiation are described in S/IP Connection Services.

If the server is not able to serve a TCP based S/IP connection the server shall send an exception to the client (see Exception handling).

4.2.1.2 Timeouts

S/IP Connections are controlled by the following two timeouts.

4.2.1.2.1 Lease timeout

The lease timeout specifies the amount of time after which the server will shutdown an idle connection to the client. This timeout exists to save resources on the server in case of loosing the link to the client without a proper shutdown of the TCP connection. The client should use S/IP Ping Service to prevent the lease timeout. Every new request from client shall restart the lease timeout at the server.

4.2.1.2.2 Busy timeout

The busy timeout specifies the amount of time within which the server should send a response to the client's request. If the server is processing the request and is not able to send the response within the busy timeout, it shall send busy responses to the client with the cycle of the busy timeout. If the client receives neither the busy response nor the response to pending request within the busy timeout, it shall shutdown the connection to the server.

Every response from server shall restart the busy timeout at the client. Figure 1 shows an example for busy response messages.

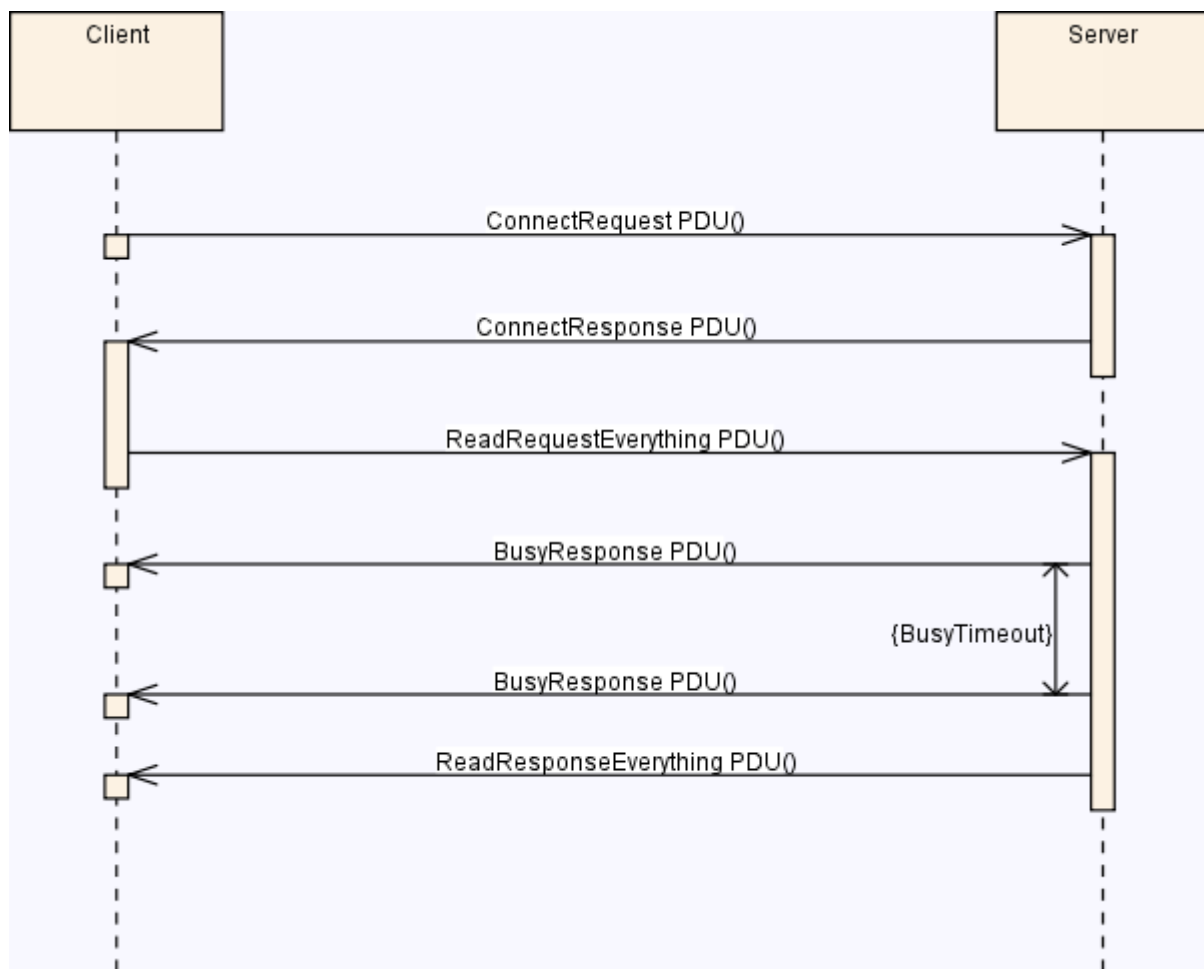


Figure 1: S/IP busy response.

4.2.1.2.2.1 Busy Response

The busy response restarts the busy timeout of an open request. The busy response looks as follows:

```

struct BusyResponse // MessageType: 68
{
}
  
```

4.2.1.3 S/IP Connection State Machines

Figure 2 shows a state machine of a S/IP connection from the clients point of view, including all states and transitions for busy- and lease-timeout, which needs to be implemented by S/IP client.

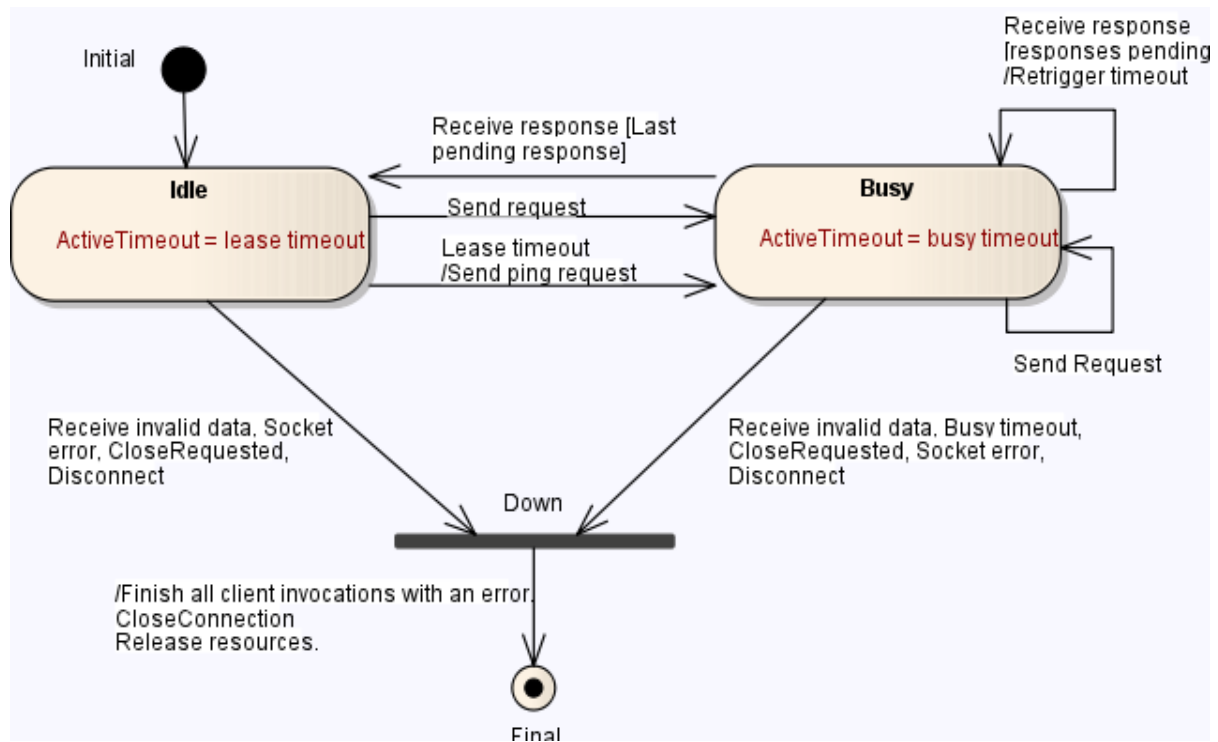


Figure 2: Client connection

Figure 3 shows a state machine of a S/IP connection from the servers point of view, including all states and transitions for busy- and lease-timeout which needs to be implemented by S/IP server.

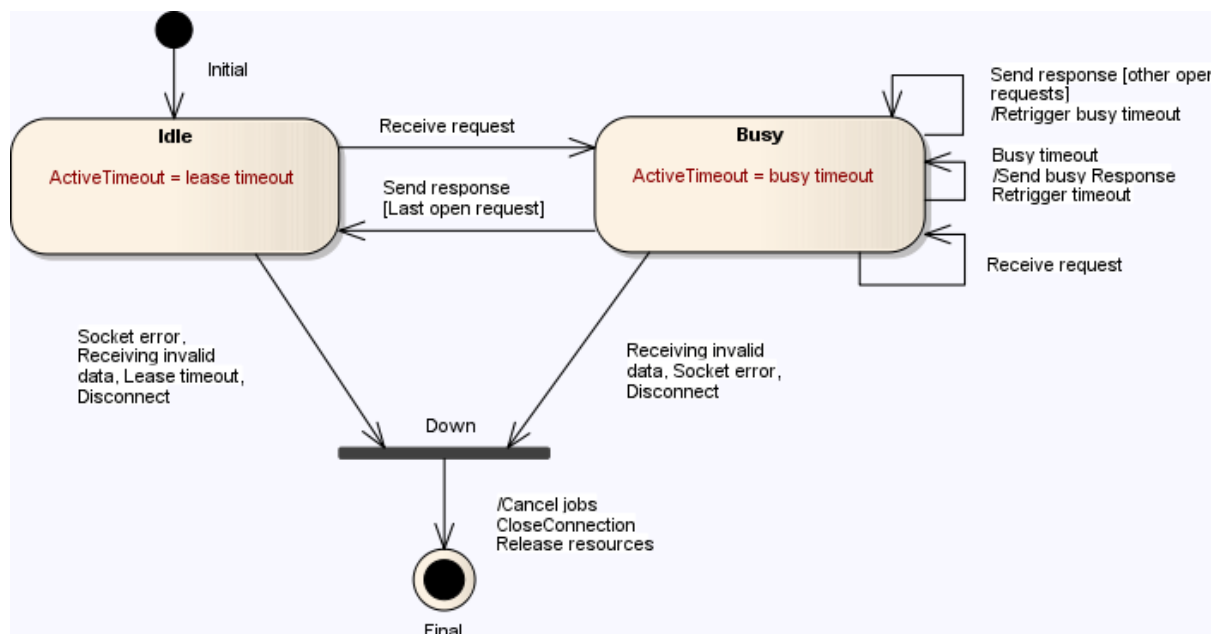


Figure 3: Server connection

4.3 TFTP

TFTP as specified in RFC 1350 shall be the standard protocol for file transfer.

The SERCOS device shall implement a TFTP server.

- The file name activates an associated action on the SERCOS device.
- The content shall be checked by the SERCOS device. If the SERCOS device is in an improper operation state, it shall return the corresponding TFTP error message. E.g. *access violation* including a meaningful error message.
- The string error-message of the TFTP protocol shall be used to transmit meaningful error messages in readable form.
- It shall be possible to upload firmware via TFTP.

It is allowed to trigger functions or commands in the device via TFTP using special file names. Functions, file names and file contents are vendor specific and not defined by SERCOS.

Note: TFTP does not support the transfer of files which are larger than 32MB.

5 Communication

Each S/IP service is defined by a request and a response message. The client can send several requests before it receives any response from the server. Furthermore the server can process incoming requests in any order.

Figure 4 shows an example of three requests answered by the server in a different order.

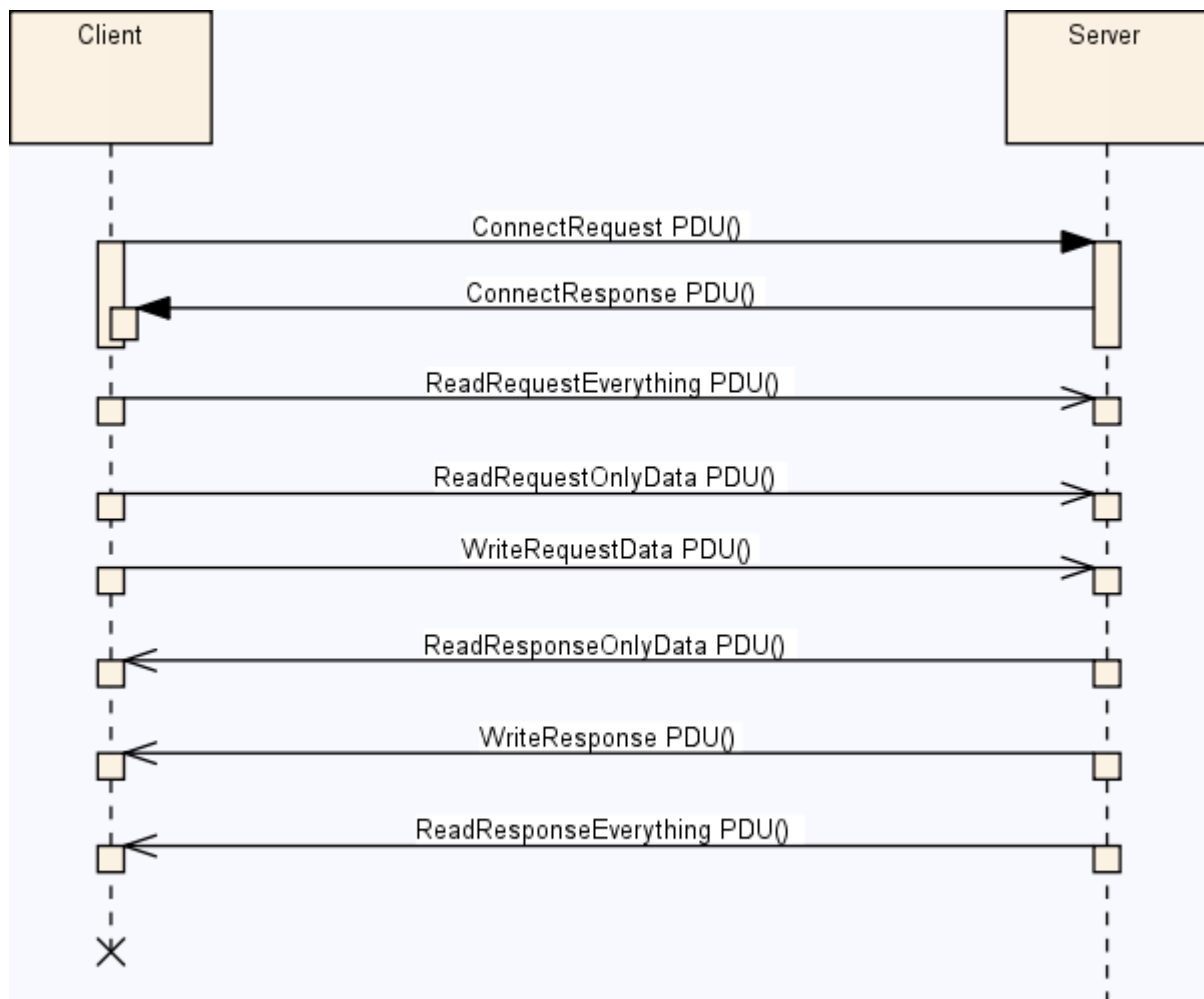


Figure 4: S/IP asynchronous requests.

If the server receives a request with an unknown message type, the request shall be ignored and an exception shall be returned to the client. Broadcast messages with an unknown message type shall not be responded with an error exception (see Exception handling).

Every request and response is composed of a S/IP Header followed by a S/IP service data unit (SDU), shown in Figure 5 .

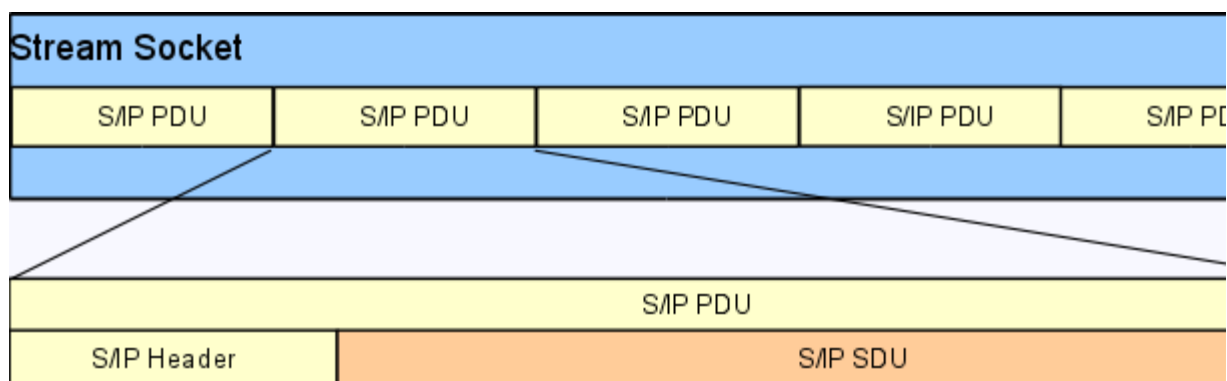


Figure 5: S/IP-PDU

5.1 S/IP Header

The S/IP header contains a transaction identifier and the message type of the PDU. The structure looks as follows.

```
struct header
{
    uint32 TransactionID;
    uint32 MessageType;
}
```

5.1.1 Transaction ID

In order to distinguish different request-response pairs in the client, the S/IP header contains a transaction identifier. This transaction identifier is an identification number which is set by the requesting client for each request. The server uses the same transaction identifier (as of request) for the corresponding response. The client uses this transaction identifier of a response to assign the response to a corresponding pending request.

5.1.2 Message Types

The message type identifies the requested service of the SERCOS devices.

The following services have been standardized by S/IP.

Table 2: Message Types

S/IP Services	Message Type Request	Message Type Response	TCP	UDP
SupportedUDPServices	61	62	-	*
Connect	63	64	*	-
Ping	65	66	*	*
ReadEverything	69	70	*	*
ReadOnlyData	71	72	*	*
ReadDescription	73	74	*	*
WriteName	75	76	*	*
WriteAttribute	77	78	*	*
WriteUnit	79	80	*	*
WriteMinMax	81	82	*	*
WriteData	83	84	*	*
WriteDataBits	85	86	*	*
ReadDataStatus	87	88	*	*
Nameplate	89	90	*	*
NameplateBroadcast	99	-	-	*
Browse	91	100	-	*
Identify	93	94	-	*
SetIp	95	96	-	*
Watchdog	101	102	*	*
Interfaces	103	104	*	*
Slaves	105	106	*	*
BrowseOnSlaveInterface	107	108	*	*
ReadSegment	109	110	*	*
Exception	-	67	*	*
Busy	-	68	*	*
Reset	97	-	-	*
Reserved	92	-	-	-

5.2.1 Exception structure

The exception message contains a common error code and an optional service specific error code.

```
struct Exception          // MessageType: 67
{
    uint16 CommonErrorCode; // see enum below
    uint32 SpecificErrorCode; // service specific error codes,
                             // e.g. error codes of service channel
}
```

5.2.2 Common Error Codes

```
enum CommonErrorCode
{
    CONNECTION_ERROR = 1, // connection can not be established
    TIMEOUT = 2,          // connection lost, timeout
    UNKNOWN_MESSAGE_TYPE = 3, // service not supported
    SERVICE_SPECIFIC = 4,    // service specific error
                             // --> see SpecificErrorCode for details
    PDU_TOO_LARGE = 5,      // request or response does not fit
                             // to the UDP datagram
                             // (limitation of PDU size)
    PDU_PROTOCOL_MISMATCH = 6 // malformed PDU
                             // e.g. received UDP datagram does not
                             // correspond to the expected PDU size
}
```

Note: For service specific error code definitions, refer to the corresponding S/IP services.

The following lists the common error codes together with the context, in which they are used.

Error Code	Context
CONNECTION_ERROR	If the server is not able to serve a TCP based S/IP connection. See TCP based communication initialization for further details.
TIMEOUT	If a timeout exceeds (see Timeouts for further details) or a TCP connection gets lost. Network activities are controlled by local timeout handling. If the server doesn't respond in time, this error code is used to indicate the error to the user on client-side.
UNKNOWN_MESSAGE_TYPE	If the server receives an unknown message type, it shall send an exception with this error code to the client. In case of a TCP based S/IP request the server returns the exception to the client and shall close the TCP stream socket connection.
SERVICE_SPECIFIC	Services are able to have their own error code. Further information are available in the SpecificErrorCode of the Exception structure. See also specific error code descriptions of the service invoked.

PDU_TOO_LARGE	This is an UDP specific error. See Limitations of PDU size for further details.
PDU_PROTOCOL_MISMATCH	This is an UDP specific error. E. g. the length of the received datagram is not conform to the expected PDU size of the service. This error indicates an incompatible implementation.

6 Services

The following sections describe the services provided by S/IP.

The information, which services are supported by a device, can be obtained by one of the following mechanisms:

- as a part of the connect response if TCP based S/IP connections are used (see S/IP Connection Services).
- the SupportedUDPServices service can be used to get a list of all UDP services supported by a device (see SupportedUDPServices Service).

6.1 S/IP Connection Services

Are used to establish and maintain a S/IP connection using TCP based S/IP.

6.1.1 Connect Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	-	-

Connect Request

In order to initiate a S/IP connection, the client sends a *ConnectRequest* PDU to the server. This request contains the desired S/IP version number and desired timeout values for the connection.

```

struct Connect          // MessageType: 63
{
    uint32 version;      // S/IP protocol version
                        // version=1 shall used for this
                        // protocol version
    uint32 busyTimeout;  // requested busy-timeout in milliseconds,
                        // the server should use
    uint32 leaseTimeout; // requested lease-timeout in milliseconds,
                        // the server should use
}

```

Connect Response

If the server is able to serve the connection, a *ConnectResponse* PDU shall be returned to the client. In the *ConnectResponsePDU*, the server provides its protocol version and the busy timeout and lease timeout values that are active for this connection to the client. If the server cannot serve the desired version of the client, it shall respond with an available version number that is lower or equal than the desired version number.

Furthermore, the *ConnectResponse* PDU contains a list of services supported by the server.

```
struct ConnectResponse          // MessageType: 64
{
    uint32  version;            // S/IP protocol version
                                // version=1 is used for this
                                // protocol version
    uint32  busyTimeout;        // busy-timeout in milliseconds,
                                // the server is using
    uint32  leaseTimeout;       // lease-timeout in milliseconds,
                                // the server is using
                                // typically 10000 - 30000 msec
    uint32  noMessageTypes;     // number of Request MessageTypes
    uint32  messagetypes[];     // The supported Request MessageTypes
                                // of the server on this TCP connection.
                                // The client must only use these
                                // message types in a request.
}
```

If the server is not able to serve the connection an *Exception* PDU shall be returned (see Connection Error).

If the timeout values in the *ConnectResponse* PDU differs from the values of the *ConnectRequest* PDU, the client is responsible to adjust the timeout values to consider the delay times and reaction times of the application.

6.1.2 Ping Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

If the server does not receive any message from the client before the lease timeout, it may close the connection to the client. The client may re-trigger the lease timeout in the server using the ping service.

Ping Request

```
struct Ping          // MessageType: 65
{
}
```

Ping Response

```
struct PingResponse // MessageType: 66  (pong)
{
}
```

6.2 Explore and IP Configuration Services

The following services are used to browse for devices within SERCOS III networks and to configure their IP settings.

6.2.1 Browse Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
-	broadcast	-

The Browse service is used to detect devices within the same subnet.

Figure 7 shows the following sequence of the Browse services.

1. A software tool (client) sends a browse request as global UDP broadcast to the local network. Within this browse request it is possible to set filters so that not all local SERCOS devices (servers) need to send a response. Possible filters are:
 - o **Master only:** Only devices those are communication masters on the local network shall respond.
 - o **Address range:** Only devices those fit the desired SERCOS address range shall respond.
2. All the SERCOS devices (servers) which receives the browse request and meets the filter condition shall send a browse response. This response contains the information like device's current network configuration, FSP types and the name that easily identifies that device.

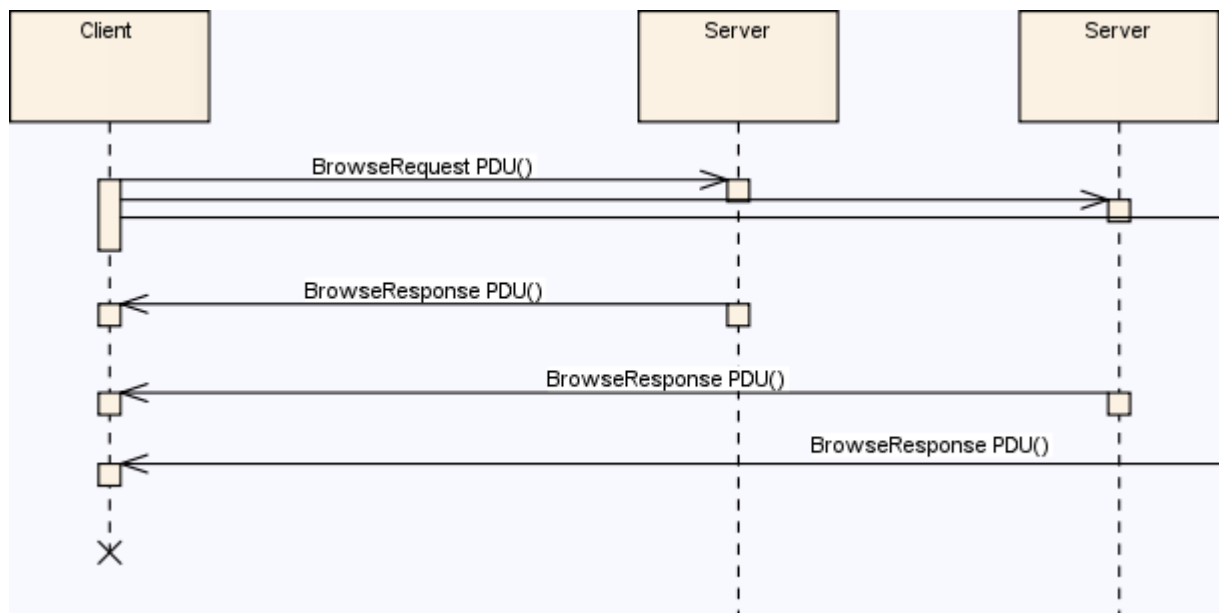


Figure 7: UDP-Browsing.

Browse Request

```

struct Browse                                     // MessageType = 91
{
    // ipaddress of the sending node.
    // The ip-address must correspond
    // to the interface of the sender
    byte  ipaddress[4];                          // [0] msb, [3] lsb
    bool  masterOnly;                            // if != 0, only devices who have
                                                // a SERCOS master interface
                                                // shall reply

    uint16 lowerSercosAddress;                   // Only devices with a SERCOS address
                                                // between lower and upper bound shall
                                                // reply. If both values are 0, all
                                                // devices shall reply. This filter
                                                // setting is not relevant for
                                                // master devices.

    uint16 upperSercosAddress;
}
  
```

Browse Response

The *BrowseResponse* depends on the interface on which the *BrowseRequest* has been received. The interface specific items are MAC Address, DHCP Features, DHCP Mode, IP Address, Subnet and Gateway.

If the implementing service doesn't know the receiving interface, it should send a response for each interface over all interfaces.

Example: Consider a device having 2 interfaces, *interface 1* and *interface 2*. In this case service should

- Send the response containing the information for *interface 1* over *interface 1* and *interface 2* and
- Send the response containing the information for *interface 2* over *interface 1* and *interface 2*.

The client (the one which sent the browse request) should check the *Node Identifier* to detect multiple responses from a node.

```

struct BrowseResponse          // MessageType: 100
{
    uint32  version;           // S/IP protocol version
                                // version=1 is used for this
                                // protocol version
    byte  node_identifier[6];   // node identifier
                                // 00:11:22:33:44:55 [0] = 00, [5] = 55
    byte  macAddress[6];        // mac address of the interface from which
                                // the response has been sent.
                                // 00:11:22:33:44:55 [0] = 00, [5] = 55
    byte  DHCPFeatures;         // bit[0]: dhcp client available
                                // bit[1]: dhcp server available
                                // bit[2]: dhcp relay agent available
    byte  DHCPMode;            // 0: no dhcp a static IP address is used
                                // 1: dhcp client active
                                // 2: dhcp server active
                                // 3: dhcp relay agent active
    byte  ipAddress[4];         // [0] msb, [3] lsb
    byte  subnet[4];            // [0] msb, [3] lsb
    byte  gateway[4];           // ip address of responsible gateway
    uint32 dplength;            // length of display name in octets
    byte  displayname[dplength]; // display name to identify the device
                                // in an browser
                                // length should not exceed 64 characters
    uint32 hnlength;            // length of hostname in octets
    byte  hostname[hnlength];   // name to identify the node S-0-1039
}

```

If a server receives a request with an invalid PDU size, it shall ignore the message and shall not send an error exception.

Note:

The element "displayname" may contain information to uniquely identify the found device. It is supposed to be directly displayed as single information in a user interface. The displayname should not exceed 64 characters. It should be language neutral.

Samples:

```

<device type> <serial number> --> "LXM62PS (12345678-0000)"
<application type> <device identification number> <sercos address> --> "AT:X-
Achse ID:8a AD:12"

```

6.2.2 SetIp Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
-	broadcast	-

The SetIp service is used to set an IP configuration of an interface.

If DHCP client is enabled ipaddress, subnet and gateway are ignored.

Figure 8 shows the sequence for the network configuration service.

1. A client sends a *set IP configuration* request as global UDP broadcast to the local network. The device to be configured is addressed using the node identifier. The request contains a complete IP configuration for the device. If the transmitted gateway address does not fit the transmitted network configuration, no default gateway shall be set (0.0.0.0). The new configuration can be set persistent or temporarily (until reboot). This is signaled by the field *persistent* in the request structure.
2. The addressed SERCOS device shall either
 - o Respond with a *set IP response* message and activate the new network configuration. The order, in which both actions are executed, is not defined. That means, the *set IP response* may be send with the old or new IP configuration.
 - o Respond with an error message as UDP broadcast. This may be done if:
 - Device is in operational mode controlled by a SERCOS master or
 - Device does not support setting a temporary network configuration.

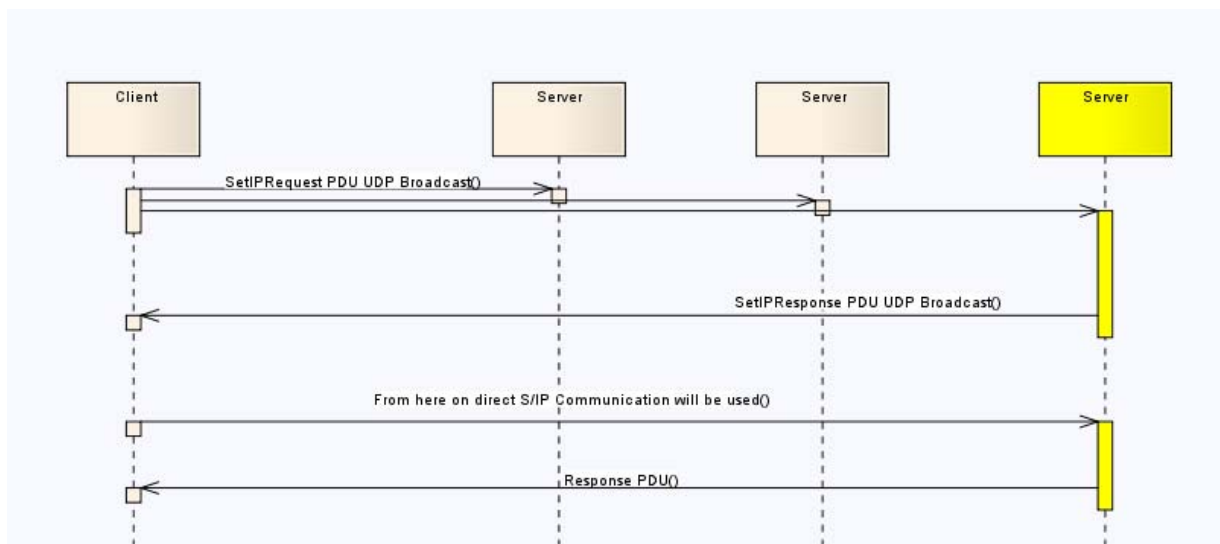


Figure 8: Sequence of setting a new network configuration on one device using UDP.

Note: Set the gateway element of the request to "0.0.0.0" to disable the gateway functionality.

Note: If the service is used for the slave interface, the settings should be reflected by the parameters:

- ipaddress (see S-0-1020 IP address)
- subnet (see S-0-1021 Subnet Mask)
- gateway (see S-0-1022 Gateway address)

SetIp Request

```

struct SetIp          // MessageType = 95
{
    byte node_identifier[6]; // 00:11:22:33:44:55 [0] = 00, [5] = 55
    byte macAddress[6];      // mac address of the interface.
                             // 00:11:22:33:44:55 [0] = 00, [5] = 55
    byte DHCPMode;          // 0: DHCP disabled
                             // 1: enable dhcp client
                             // 2: enable dhcp server
                             // 3: enable dhcp relay agent
    byte ipAddress[4];      // [0] msb, [3] lsb
                             // new subnet mask
    byte subnet[4];         // [0] msb, [3] lsb
                             // default gateway to forward ip messages
    byte gateway[4];        // [0] msb, [3] lsb
    bool persistent;        // if != 0, store settings persistent
    uint32 hnlength;        // length of hostname in octets
    byte hostname[hnlength]; // name to identify the node (see S-0-1039)
}

```

SetIp Response

```

struct SetIpResponse // MessageType = 96
{
}

```

In case of an error an Exception SDU containing an error code will be sent.

```

enum SetIpErrorCode
{
    INVALID_DEVICE_STATE = 1 // invalid device state
                             // (IP configuration can not be activated)
    TEMP_IP_NOT_SUPPORTED = 2 // temporary IP configuration is
                             // not supported
    INVALID_SERVICE_DATA = 3 // The given data are incorrect
    SET_IP_NOT_SUPPORTED = 4 // setting of static IP_ address
                             // is not supported
    INVALID_DHCP_MODE = 5    // device does not support DHCP_MODE
}

```

6.2.3 Interfaces Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

This service gets all the interfaces of a node.

Interfaces Request

```

struct Interfaces // MessageType = 103
{
}

```

Interfaces Response

```

struct InterfacesResponse      // MessageType = 104
{
    byte node_identifier[6];    // 00:11:22:33:44:55 [0] = 00, [5] = 55

    uint32 dplength;           // length of display name in octets
    byte displayname[dplength]; // display name to identify the device
                                // in an browser, see BrowseResponse
                                // length should not exceed 64 characters

    uint32 noInterfaces;       // number of interfaces
    struct
    {
        byte macAddress[6];    // mac address of the interface
                                // 00:11:22:33:44:55 [0] = 00, [5] = 55
        byte ipAddress[4];     // [0] msb, [3] lsb ipaddress
                                // of the slave interface
        byte interfaceType;    // 0: an Ethernet interface,
                                // e. g. engineering port
                                // 1: a SERCOS Slave interface
                                // 2: a SERCOS Master interface
    } interfaces[noInterfaces];
}

```

6.2.4 Slaves Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

This service gets all slaves connected to a master interface. Only SERCOS masters have to implement this service.

Slaves Request

```

struct Slaves                  // MessageType = 105
{
    byte macAddress[6];        // mac address of the master interface
                                // 00:11:22:33:44:55 [0] = 00, [5] = 55
}

```

Slaves Response

```

struct SlavesResponse          // MessageType = 106
{
    uint32 noSlaves;           // number of slaves
    struct
    {
        byte ipAddress[4];    // [0] msb, [3] lsb ipaddress of the slave
        byte node_identifier[6]; // 00:11:22:33:44:55 [0] = 00, [5] = 55
    } slaves[noSlaves];
}

```

In case of an error an Exception SDU containing an error code will be sent.

```
enum SlavesErrorCode
{
    INVALID_MAC_ADDRESS = 1      // Interface MAC address is not valid
                                // for Slaves service
}
```

6.2.5 BrowseOnSlaveInterface Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

This service requests the master to browse on another slave interface for masters. Only SERCOS masters have to implement this service. This service allows a slave of ring1 to get information from its master (ring1) from another ring (ring2). The node, master of ring1, must be a slave of ring2. Browse responses are collected by the server for a duration of timeToBrowse. After timeToBrowse the server will report the collected responses to the client.

BrowseOnSlaveInterface Request

```
struct BrowseOnSlaveInterface      // MessageType = 107
{
    byte macAddress[6];            // macAddress of the slave interface
                                   // to browse for masters
    int32 timeToBrowse;            // time to wait for responses in
                                   // milliseconds
}
```

BrowseOnSlaveInterface Response

```
struct BrowseOnSlaveInterfaceResponse // MessageType = 108
{
    uint32 noMasters;              // number of masters
    struct
    {
        byte node_identifier[6];   // master node identifier
                                   // 00:11:22:33:44:55 [0] = 00, [5] = 55
        byte ipaddress[4];         // master ip address
                                   // [0] msb, [3] lsb ipaddress
                                   // of the slave
    } masters[noMasters];
}
```

In case of an error an Exception SDU containing an error code will be sent.

```
enum BrowseOnSlaveInterfaceErrorCode
{
    INVALID_MAC_ADDRESS = 1      // Interface MAC address is not valid
                                // for Slaves service
}
```

6.3 Identification Services

The following services allow the identification of S/IP devices.

6.3.1 Identify Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
-	broadcast	-

The Identify service is used for the identification of a device, by instructing the device to identify itself, e.g. blinking.

Figure 9 shows the sequence of the UDP identify service.

1. A client sends an identify request as global UDP broadcast to the local network. The device to be identified is addressed using the node identifier in the identify request.
2. The addressed SERCOS device shall identify itself, the device signals for 4 seconds. After that, signaling is stopped automatically. The signaling LED behavior is defined in the section SERCOS_III_LED. Identified device shall also send an identify response.

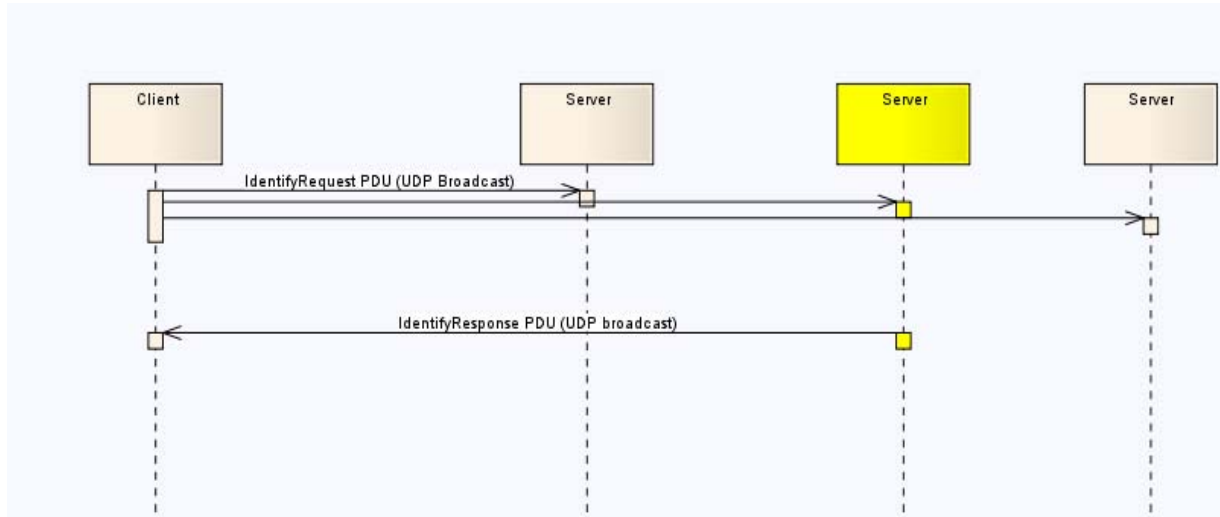


Figure 9: UDP-Identification.

Identify Request

```

struct Identify          // MessageType = 93
{
    // node identifier of the device
    byte node_identifier[6]; // 00:11:22:33:44:55 [0] = 00, [5] = 55
}
  
```

Identify Response

Only the addressed device sends a response.

```
struct IdentifyResponse    // MessageType = 94
{
}
```

6.3.2 Nameplate Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

The nameplate service requests the electronic nameplate of the node. The nameplate information is returned as a list of nameplate entries. Each entry contains a leading id and a string value. The ids are defined in table Table 4. If not explicitly defined, the display format of the string value is given by the attribute of the referred IDN.

Nameplate Request

```
struct Nameplate          // MessageType = 89
{
}
```

Nameplate Response

```
struct NameplateResponse  // MessageType = 90
{
    uint32 noEntries;      // number of nameplate entries
    struct
    {
        byte id;           // id of the nameplate entry
        uint32 length;     // length of value in octets
        byte[length] value; // value of the nameplate entry:
                           // string Variable-Len
    } nameplate_entries[noEntries];
}
```

Table 4: Nameplate IDs

ID	Name	Comments
1	reserved	
2	Vendor Name	refers to S-0-1300.x.02 Vendor Name
3	SERCOS III Vendor Code	refers to S-0-1300.x.03 Vendor Code
4	Device Name	refers to S-0-1300.x.04 Device Name
5	Vendor Device ID	refers to S-0-1300.x.05 Vendor Device ID
6	reserved	
7	Function-Revision	refers to S-0-1300.x.07 Function Revision

8	HW-Revision	refers to S-0-1300.x.08 Hardware Revision
9	FW-Revision	refers to S-0-1300.x.09 Software Revision
10	Firmware Loader Revision	refers to S-0-1300.x.10 Firmware Loader Revision
11	reserved	
12	Serialnumber	refers to S-0-1300.x.12 Serial Number
13	Manufacturing Date	refers to S-0-1300.x.13 Manufacturing Date
14 .. 19	reserved	
20	Operational Hours	refers to S-0-1300.x.20 Operational Hours
21 .. 31	reserved	
32	Hostname	refers to S-0-1039 Hostname
33	Quantity of slaves	refers to the number of list elements of S-0-1046 List of SERCOS addresses in device the display format shall unsigned decimal
34..127	reserved	
128..255	manufacturer specific	

6.3.3 NameplateBroadcast Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
-	broadcast	-

The NameplateBroadcast service is used to gather information about a device.

The *NameplateResponse* returns a list of IDN parameters. It may be used for additional info about devices without defined communication settings.

```
struct NameplateBroadcast // MessageType = 99
{
    // node identifier of the device
    byte node_identifier[6]; // 00:11:22:33:44:55 [0] = 00, [5] = 55
}
```

The response is defined in Nameplate Service.

6.4 SERCOS Parameter Access

In most cases SERCOS device will have only one interface and hence only one IP address. To access the parameters (reading and writing parameters) from different slaves of a device, slaves need to be addressed in the request. This addressing will be done using *slave index*. The *slave-index* is an internal numeration of all slaves in a device.

The number of slaves in one device corresponds to the number of elements in the global parameter S-0-1046. Absence of parameter S-0-1046 indicates that the device has exactly one slave. In this case zero shall be used for *slave index*. Global parameters are available with any *slave index*.

A device may define additional parameters using the SERCOS parameter structure. These parameters are accessible by the *SlaveExtension*. If not used, the *SlaveExtension* should be zero.

6.4.1 Parameter data

The coding of the parameter data including the min and max value corresponds to the coding of the service channel with some exceptions described below:

- Coding is done in little endian
- The length of the value is given with the attribute size of (1,2,4,8) in octets.
- Parameter with string as data uses lists of 1 byte elements. The length of the parameter data represents the coding length of the string in octets, not the number of characters. There is no terminating 0 byte. Strings are coded in utf-8.
- Lists
The list header which contains the maximum length and the current length is not part of the S/IP data. Only list element value data's are part of the transmitted information. The current length of the list will be returned by the datalength. The number of elements in the list may be evaluated using the element size information of the attribute and the datalength.
Use ReadEverything or ReadDescription to get the maximum available list size in octets.

6.4.2 IDNs

IDNs are passed in the services to address a specific parameter. IDNs are also part of the parameter value, if the parameter display format is "IDN". See the IDN specification how to code and decode the IDN.

6.4.3 Parameter Service Specific Error Codes

The following enumeration defines the service specific error codes for S/IP. See also the SVC error codes

```
enum SipErrorCode
{
    // 0x0 - 0x7FFFFFFF                // reserved for SVC Error Codes,
                                        // e.g. 0x1001 No IDN
    // > 0x80000000                    // Addition Error Codes
    SLAVE_INDEX_INVALID                = 0x80000001,
    PARAMETER_INVALID                  = 0x80000002, // can be used to avoid
                                                // invalid access;
                                                // e.g. write access
                                                // to procedure commands
    PARAMETER_NOT_ACCESSIBLE = 0x80000003, // Parameter cannot be accessed
                                                // e.g. if parameter database
                                                // is not ready
}
```

6.4.4 ReadEverything Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

ReadEverything Request

```
struct ReadEverything // MessageType: 69
{
    uint16 SlaveIndex;
    uint16 SlaveExtension;
    uint32 IDN;
}
```

ReadEverything Response

```

struct ReadEverythingResponse // MessageType: 70
{
    uint16    validelements; // bitmask of the following elements,
                            // which are valid in this response
                            // invalid length elements shall be set to 0

                            // bitmask | description
                            // -----
    ----
    uint16    data_status;   // 0x01    | data valid & command
                            //          | acknowledge ref. IDN numbers
    uint16    namelength;    // 0x02    | length of datablock element
                            //          | 2 in octets
    uint32    attribute;     // 0x04    | datablock element 3
    uint16    unitlength;    // 0x08    | length of datablock element
                            //          | 4 in octets
    byte[8]   min;           // 0x10    | datablock element 5
    byte[8]   max;           // 0x20    | datablock element 6
    uint32    maxlistlength; // --      | maximum length of datablock
                            //          | element 7
                            //          | in octets. Valid if
                            //          | parameter is a list
    uint32    datalength;    // 0x40    | length of datablock
                            //          | element 7 in octets
    byte[namelength] name;   // 0x02    | name of parameter;
                            //          | string Variable-Len
    byte[unitlength] unit;   // 0x08    | unit of parameter;
                            //          | string Variable-Len
    byte[datalength] data;   // 0x40    | datablock element 7
}

```

Note: The response has to contain all implemented elements of the IDN.

6.4.5 ReadOnlyData Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

ReadOnlyData Request

```
struct ReadOnlyData          // MessageType: 71
{
    uint16 SlaveIndex;
    uint16 SlaveExtension;
    uint32 IDN;
}
```

ReadOnlyData Response

```
struct ReadOnlyDataResponse  // MessageType: 72
{
    uint32 attribute;
    uint32 length;            // length of data counted in octets
    byte[length] data;
}
```

6.4.6 ReadDescription Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

ReadDescription Request

```
struct ReadDescription          // MessageType: 73
{
    uint16 SlaveIndex;
    uint16 SlaveExtension;
    uint32 IDN;
}
```

ReadDescription Response

```
struct ReadDescriptionResponse // MessageType: 74
{
    uint16  validelements; // bitmask of the following elements,
                           // which are valid in this response
                           // invalid length elements shall be set to 0

                           // bitmask | description
                           // -----
    ----
    uint16  namelength;    // 0x02    | length of datablock
                           // element 2 in octets
    uint32  attribute;     // 0x04    | datablock element 3
    uint16  unitlength;    // 0x08    | length of datablock
                           // element 4 in octets
    byte[8] min;           // 0x10    | datablock element 5
    byte[8] max;           // 0x20    | datablock element 6
    uint32  maxlistlength; // --      | maximum length of
                           // datablock element 7
                           //          | in octets. Valid if
                           //          | parameter is a list
    byte[namelength] name; // 0x02    | name of parameter;
                           // string Variable-Len
    byte[unitlength] unit; // 0x08    | unit of parameter;
                           // string Variable-Len
}
```

Note: The response has to contain all implemented elements of the IDN.

6.4.7 ReadDataStatus Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

ReadDataStatus Request

```
struct ReadDataStatus          // MessageType: 87
{
    uint16 SlaveIndex;
    uint16 SlaveExtension;
    uint32 IDN;
}
```

ReadDataStatus Response

```
struct ReadDataStatusResponse // MessageType: 88
{
    uint16 data_status;          // data valid & command acknowledge
                                // refer to the IDN numbers
}
```

6.4.8 ReadSegment Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

This service reads operation data elements. The response length depends on the data type.

ReadSegment Request

```

struct ReadSegment                                // MessageType: 109
{
    uint16 SlaveIndex;
    uint16 SlaveExtension;
    uint32 IDN;
    uint16 StartIndex;                            // 0 based index of the first
                                                    // list element
    uint16 NumberOfRequestedElements;             // requested number of list elements
}

```

ReadSegment Response

```

struct ReadSegmentResponse                        // MessageType: 110
{
    uint32 attribute;
    uint16 NumberOfAvailableElements;             // Number of list elements
    uint32 length;                                // length of data counted
                                                    // in octets
    byte[length] data;                            // parameter data
}

```

Note: The device can return fewer elements than requested. Response having fewer elements indicates that the MTU size can be too small or the end of the list is reached.

Note: If the start index of the request is out of range, the server shall reply with the data length of zero.

Note: IDNs with a fix length shall be treated as list IDNs with length 1.

6.4.9 WriteName Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

WriteName Request

```
struct WriteName // MessageTypes: 75
{
    uint16 SlaveIndex;
    uint16 SlaveExtension;
    uint32 IDN;
    uint16 namelength;
    byte[namelength] name; // string Variable-len
}
```

WriteName Response

```
struct WriteNameResponse // MessageTypes: 76
{
    // empty response;
    // data was written correctly
    // in case of an error, an exception will be
    // sent instead
}
```

6.4.10 WriteAttribute Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

WriteAttribute Request

```
struct WriteAttribute // MessageType: 77
{
    uint16 SlaveIndex;
    uint16 SlaveExtension;
    uint32 IDN;
    uint32 attribute;
}
```

WriteAttribute Response

```
struct WriteAttributeResponse // MessageTypes: 78
{
    // empty response;
    // data was written correctly
    // in case of an error, an exception will be
    // sent instead
}
```


6.4.11 WriteUnit Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

WriteUnit Request

```
struct WriteUnit          // MessageType: 79
{
    uint16      SlaveIndex;
    uint16      SlaveExtension;
    uint32      IDN;
    uint16      unitlength;
    byte[unitlength] unit;    // string with var-len
}
```

WriteUnit Response

```
struct WriteUnitResponse  // MessageType: 80
{
    // empty response; data was written correctly
    // in case of an error, an exception will be
    // sent instead
}
```

6.4.12 WriteMinMax Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

WriteMinMax Request

```
struct WriteMinMax        // MessageType: 81
{
    uint16  SlaveIndex;
    uint16  SlaveExtension;
    uint32  IDN;
    byte[8] min;          // datablock element 5
    byte[8] max;          // datablock element 6
}
```

WriteMinMax Response

```
struct WriteMinMaxResponse // MessageType: 82
{
    // empty response; data was written correctly
    // in case of an error, an exception will be
    // sent instead
}
```

6.4.13 WriteData Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

WriteData Request

```
struct WriteData          // MessageType: 83
{
    uint16  SlaveIndex;
    uint16  SlaveExtension;
    uint32  IDN;
    uint32  length;        // length of data counted in octets
    byte[length] data;
}
```

WriteData Response

```
struct WriteDataResponse  // MessageType: 84
{
    // empty response; data was written correctly
    // in case of an error, an exception will be
    // sent instead
}
```

6.4.14 WriteDataBits Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

WriteDataBits Request

```
struct WriteDataBits          // MessageType: 85
{
    uint16  SlaveIndex;
    uint16  SlaveExtension;
    uint32  IDN;
    uint32  length;           // length of data and mask counted in octets
    byte[length] data;        // value of bits to be modified
    byte[length] dataMask;    // bits to be modified
}
```

Note: If the length sent by the client is less than the size of the parameter, the server shall extend the MSB of mask and value with 0 octets.

Note: WriteDataBits is supported only for non list parameters.

WriteDataBits Response

```
struct WriteDataBitsResponse // MessageType: 86
{
    // empty response; data was written
    // correctly in case of an error, an
    // exception will be sent instead
}
```

6.5 Device Management Services

6.5.1 Reset Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
-	broadcast	-

The Reset service is used to reset one or more devices in a network.

Since the reset service is going to interrupt the communication, the reset has to be delayed by the server. The devices which shall participate in the reset are identified by the *node_identifiers*. As soon as a reset request has been received at a device, it shall start an internal timer and after the timeout time has elapsed, reset will be performed. Because of possible data losses, it is recommended that the client repeats the request up to five times with a gap of ~100 ms between the requests. If a server receives a reset request twice, the timer

shall be restarted. If there is any situation which does not allow the execution of the reset request, e.g. safety conditions, the reset request shall be ignored by the device.

The maximum number of devices which can be addressed by a reset request is restricted to 85. This is because of restriction on UDP packet size. If there are more than 85 devices which should be restarted, it is possible to send different reset requests consecutively.

This service shall perform a cold reset. The main purpose of this service is activating the depending update files, transferred e.g. via TFTP.

Note: Devices will not send any response to this request.

Reset Request

```

struct Reset                                     // MessageType = 97
{
    uint32 timeout;                             // waiting time
                                                // in milliseconds
                                                // until reset is
                                                // performed by the device
    uint16 no_node_identifiers;                 // number of
                                                // node_identifiers
                                                // inside range
                                                // from 0 to 85.
                                                // If 0, no device shall
                                                // perform a reset,
                                                // else the amount
                                                // of node_identifiers
    byte node_identifier[no_node_identifiers][6]; // Array of
                                                // node_identifiers:
                                                // 00:11:22:33:44:55 [0]
                                                // = 00, [5] = 55
}

```

The following Figure 10 shows the usage of UDP *ResetRequest*. A client tool is used to send two reset requests. The first request intends to reset the first and the third device. After the request has been sent, the user wants to reset the second device, too. Therefore, the client tool send a second request which addresses the second device only. After the timeout time has elapsed, the devices perform the hardware reset.

Note that the Figure 10 does not show the recommended repetition of the sent requests.

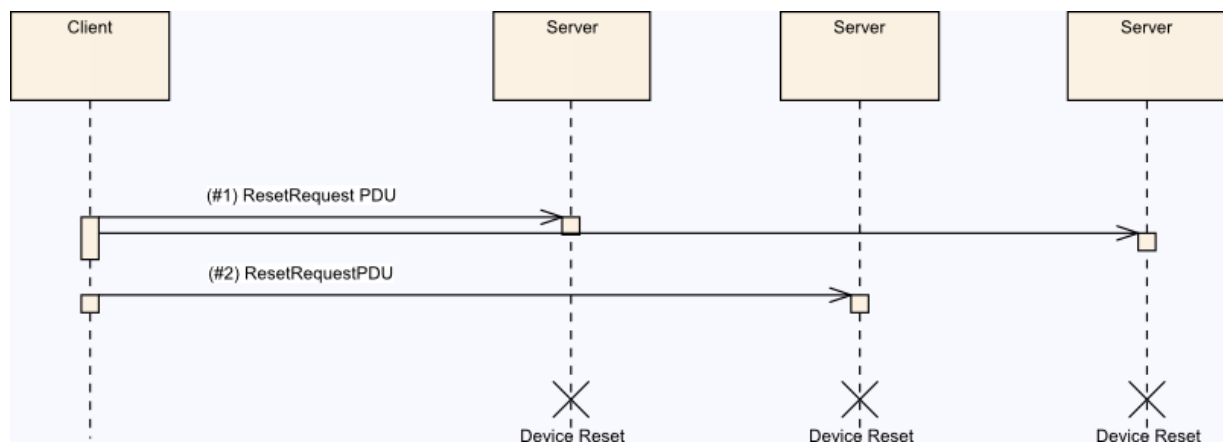


Figure 10: Usage UDP Reset Request

6.5.2 TFTP Firmware Management / Update

This service can be used via the following transport protocols:

TCP	UDP	TFTP
-	-	X

- Firmware management uses the TFTP protocol.
- Any error shall be responded using the TFTP error messages e.g. *wrong firmware version* including a describing error message. TFTP error messages shall not be used for a successful operation.

Note: An overall activation of a new firmware shall be invoked using the reset service.

6.6 Other Services

6.6.1 SupportedUDPServices Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
-	unicast	-

This service returns a list of all supported "UDP based S/IP" services (unicast and broadcast).

SupportedUDPServices Request

```

struct SupportedUDPServices           // MessageType: 61
{
}
  
```

SupportedUDPServices Response

```
struct SupportedUDPServicesResponse // MessageType: 62
{
    uint32 noMessageTypes;           // number of Request MessageTypes
    uint32 messagetypes[];           // The supported request
                                     // MessageTypes of the server
                                     // The client shall only use
                                     // these message types in a request.
}
```

6.6.2 Watchdog Service

This service can be used via the following transport protocols:

TCP	UDP	TFTP
x	unicast	-

This service implements a *keep alive* mechanism for an activity. This can be used to ensure an "in time termination" of specific activities of the device, started by the client.

While the specific activity is being executed by the device, the client shall keep on triggering the watchdog to keep the activity running. With this mechanism, device will come to know that the client is still alive and observing the activity being executed. If the device does not receive watchdog request before the timeout, due to any communication problems, the device will stop the specific activity automatically. Refer Figure 11 for details.

Example: This service should be used to terminate an axis movement initiated by S/IP client.

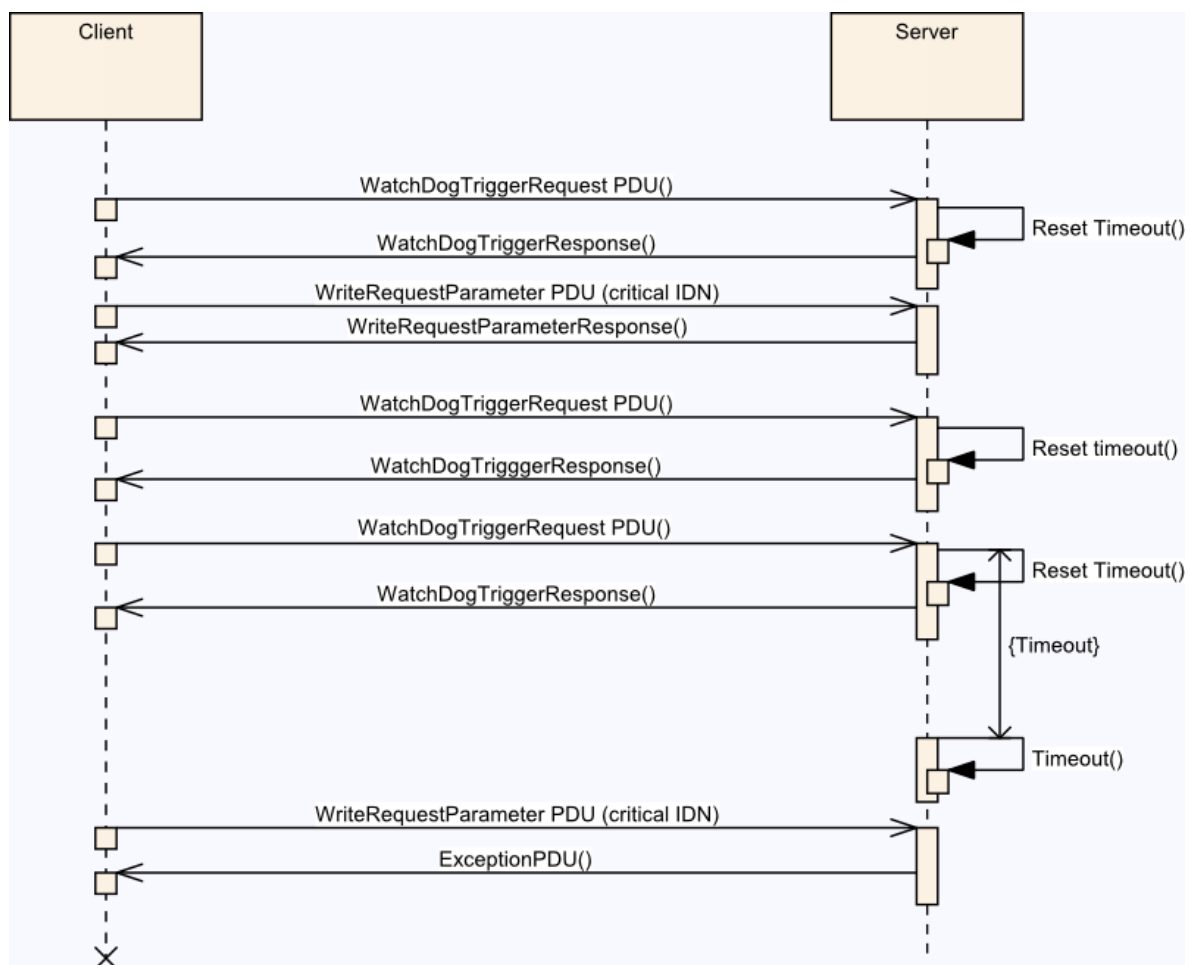


Figure 11: Sequence for watchdog trigger service and client application timeout.

This service message shall always be transmitted if an axis is moved using parameter access over S/IP. If this service message is not transmitted by the server for a dedicated time, a watchdog shall trigger and put the device into a safe state.

Since S/IP is only there to transport the data, functional safety can not be provided by S/IP when moving axes. The definition of a *safe state* as well as the timeout values are dependent on the device and can not be specified by S/IP.

As an example, critical IDNs like S-0-0134 on drives shall only be writable, as long as the watchdog has not expired. After the watchdog has expired which is the initial state of the watchdog, all critical IDNs shall be reset to fall back values. Every device needs to know its critical IDNs by itself. If the watchdog has expired and a critical IDN should be written, the SVC error message 0x700c, „*Operation data is write protected, due to other settings. (e.g., parameter, operation mode, drive enable, drive on etc.)*“, shall be responded.

The request message received by a device resets the devices watchdog. The device will reply to the request with a response message including the expiration period of the watchdog in milliseconds, refers to Watchdog trigger service.

Watchdog Request

```
struct WatchdogTrigger          // MessageType: 101
{
}
```

Watchdog Response

```
struct WatchdogTriggerResponse // MessageType: 102
{
    uint32 timeout;             // in milliseconds
}
```


7 Classification

SERCOS III defines several S/IP classes that may be implemented by the devices.

S/IP defines several classes. If a device supports a class, all services of that class shall be implemented. The following tables show the mandatory services for master and slaves. Table 5 shows the defined Inter protocol classes.

Table 5: IPS classes

Class Ids	Class name	comments
1	TCP Basic	
2	UDP Basic	
3	Device Management	
4	Explore & IP Configuration Services	
5	SERCOS Parameter access	

At least one of the classes TCP Basic or UDP Basic shall be implemented.

Table 6: Class TCP Basic

Service	if UDP is supported	if TCP is supported
Connect	-	x
Ping	-	x
Busy	-	x
Nameplate	-	x

Table 7: Class UDP Basic

Service	if UDP is supported	if TCP is supported
Nameplate	x	-
SupportedUdpServices	x	-

Table 8: Class Device Management

Service	if UDP is supported	if TCP is supported
TFTP Firmware Update	x	x

Table 9: Class Explore & IP Configuration Services

Service	if UDP is supported	if TCP is supported
Browse	x	-
SetIpConfiguration	x	-
Identify	x	-

Table 10: Class SERCOS Parameter access

Service	if UDP is supported	if TCP is supported
ReadEverything	x	x
ReadOnlyData	x	x
ReadDescription	x	x
ReadDataStatus		x
ReadSegment	x	
WriteData	x	x
WriteDataBits	x	x

8 Appendix A

Within this chapter use cases of S/IP browsing are explained.

8.1 Local Browsing

Local browsing is used to explore and configure communication settings of devices, which can be reached using Ethernet broadcasts. All requests are global UDP broadcasts (IP-address 255.255.255.255). Responses in the same network may be sent as unicast while responses in the other networks shall be sent as broadcast.

Local browsing is implemented by the following services.

- Browse Service
- Identify Service
- SetIp Service.

An example for local browsing, device identification and network configuration is explained (using flowchart) in Figure 12.

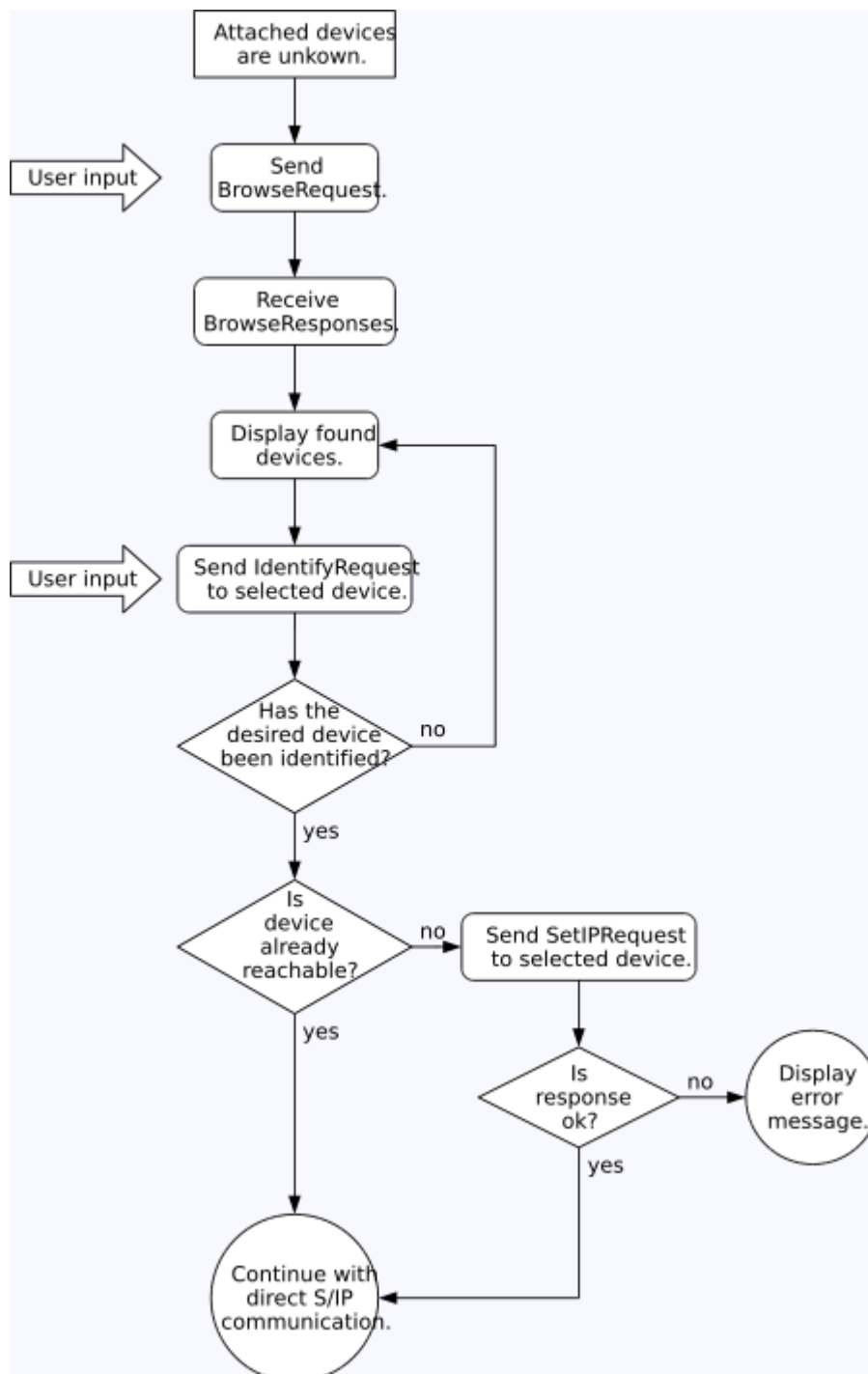


Figure 12: Example flowchart for UDP browsing

As a first step a browse request is sent on the local network to find out all SERCOS devices. All responses to the browse request are then displayed to the user. When the user selects one of the found devices, an identification request is sent to that SERCOS device, which then shall identify itself by flashing its SERCOS III LED. If the identified device is not the desired one, the user can then select another device from the list. As soon as the desired device is identified, a configuration request can be sent to the device to assign a correct network configuration. After this process, other S/IP services can be used to communicate with the SERCOS device.

Figure 13 shows S/IP messages corresponding to the process explained above.

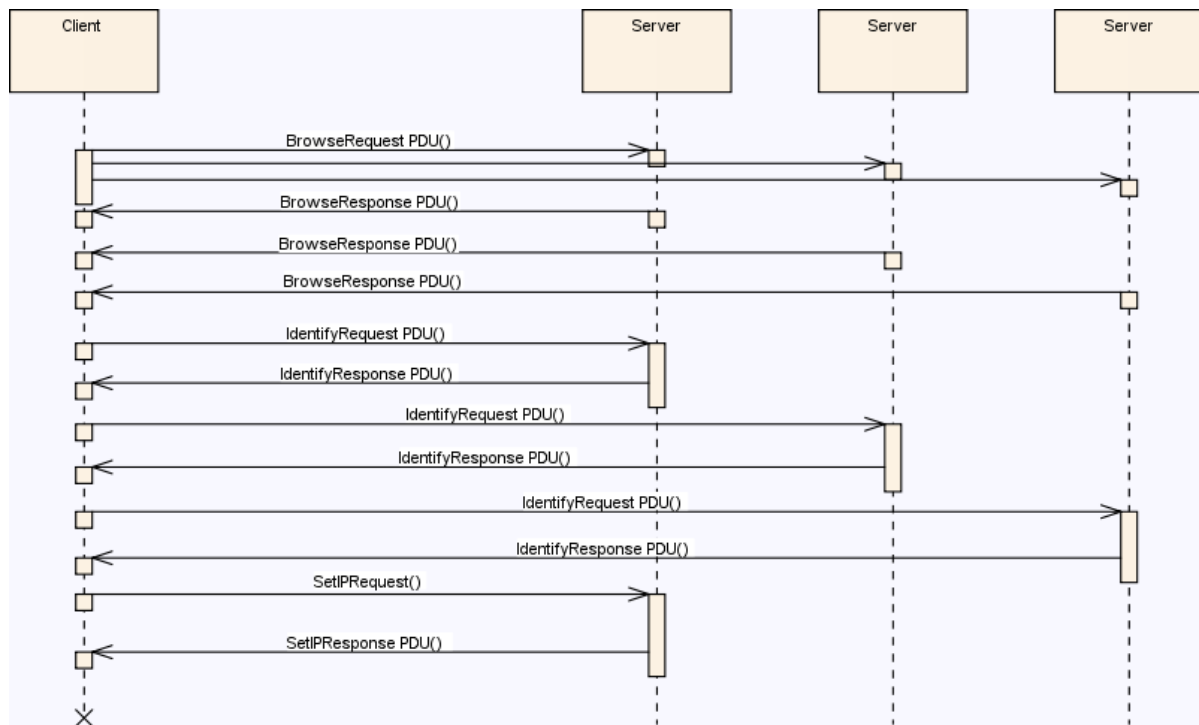


Figure 13: Example of complete sequence of UDP browsing.

8.2 Remote Browsing

The term *remote* is used with respect to a browsing client connected to a network within hierarchical SERCOS III topology. As shown in Figure 14, the network to which a browsing PC is connected is referred to as *local network* and all other networks are referred to as *remote networks*.

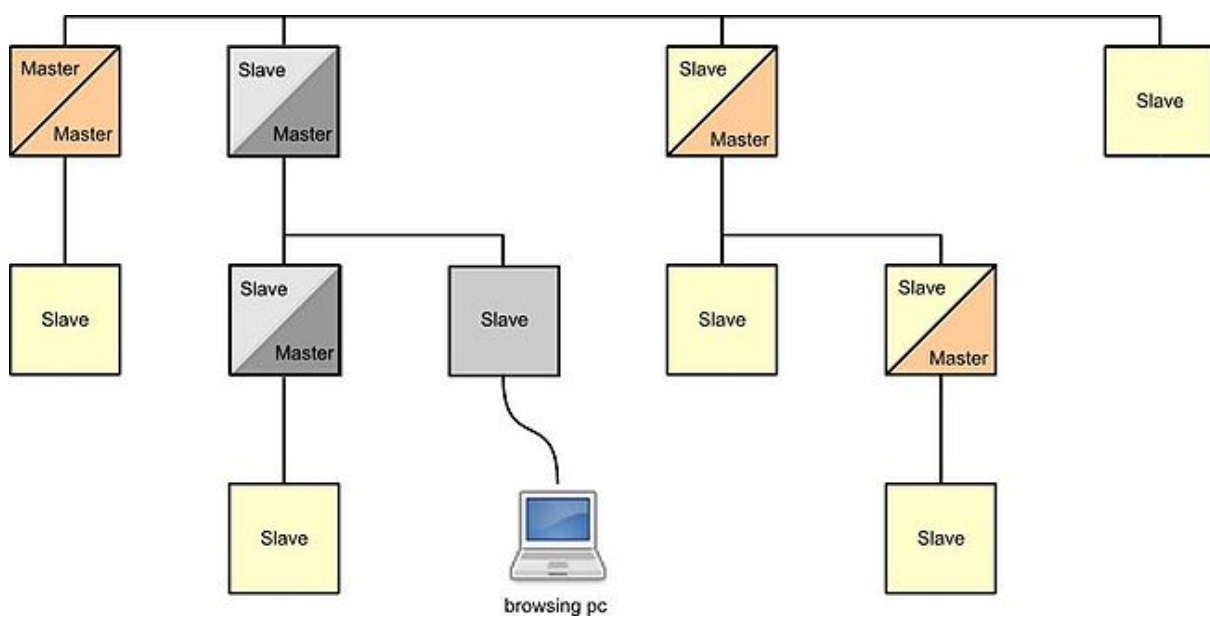


Figure 14: Example for Remote Browsing with Routing Topology

Remote browsing services are required to browse the devices attached to a remote network. *Remote browsing* is implemented using following services.

- Interfaces Service
- Slaves Service
- BrowseOnSlaveInterface Service

Figure 15 shows an example flowchart for *remote browsing* within the configured SERCOS network.

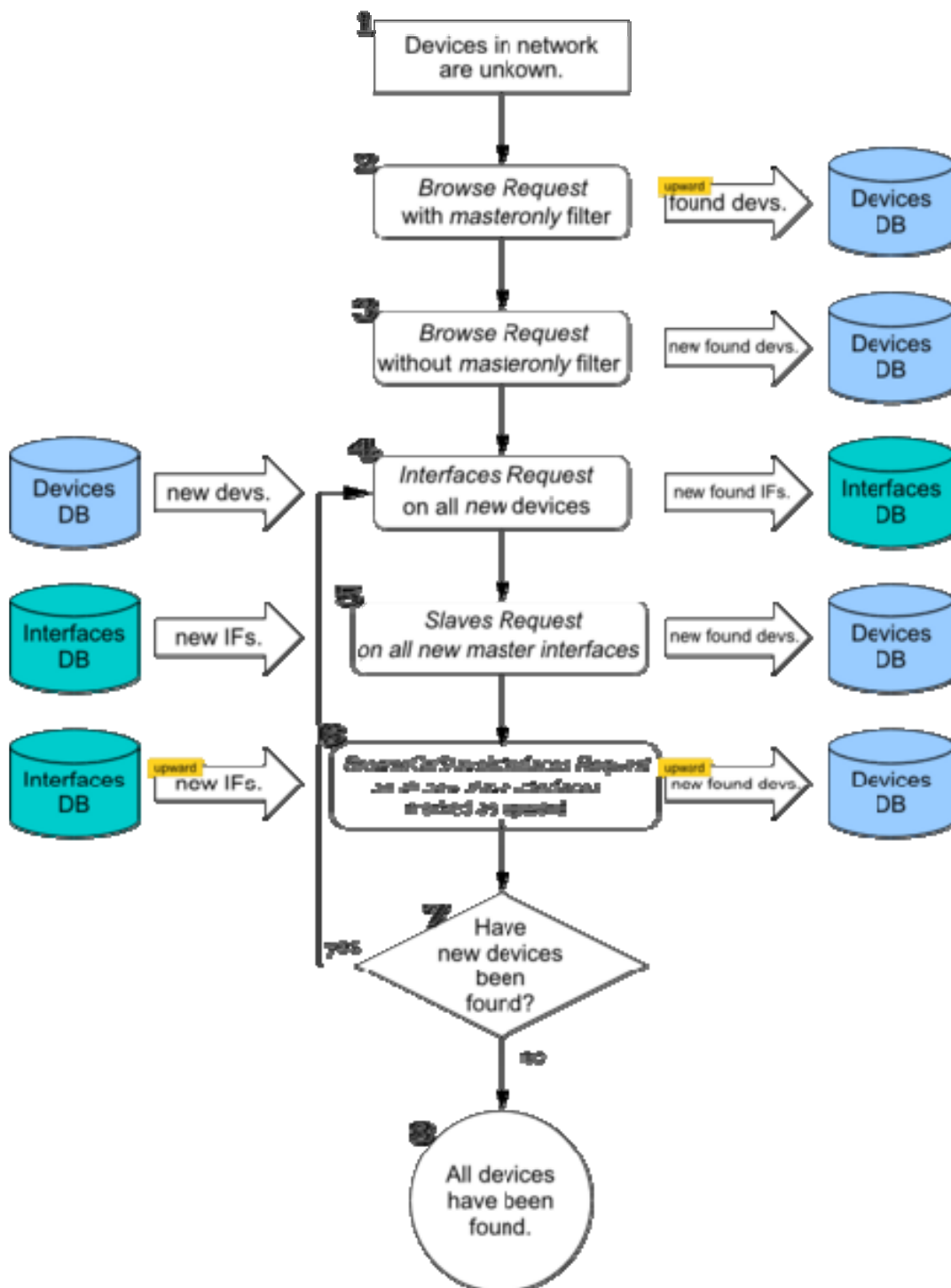


Figure 15: Flowchart for remote browsing

In this example Interfaces Service is used to get all interfaces and its types of a device.

Subsequently calls of Slaves Service on an interface of type 'master' returns all slaves that are attached to this interface.

In contrast if a master has also one or more slave interfaces a call of BrowseOnSlaveInterface Service on one of its interfaces of type 'slave' returns all masters attached to that interface.

The tag *upward* marks devices which are browsed from a master interface side. This devices may have slave interfaces, where further masters and slaves are accessible. In this case, these devices mark an upward path within the hierarchical SERCOS III topology.

The tag *new* marks a device or an interface as new in the database and not currently handled.

The shown steps of Figure 15 can be explained as follows:

1. **Initial state**

To start with, a browsing PC is connected to the SERCOS III network. No devices are known to the PC at this point.

2. **Find local masters**

Browsing PC needs to find attached master devices. So it sends a *Browse Request* with the *masterOnly* filter set. The responses shall be added to a *Devices DB* and marked with the tag *upward* and *new*.

3. **Find local slaves [optional]**

The resting slave devices can be found by sending the *Browse Request* but without the *masterOnly* filter set. The responses includes slaves and also master devices. Only found new slave devices shall be added to a *Devices DB* and marked with the tag *new*.

4. **Get all interfaces of new devices**

All devices marked as *new* in the *Devices DB* shall be scanned with the *Interfaces Request*. The tag *new* shall then be removed from the scanned devices in the *Devices DB*. All found interfaces that are not yet present in the *Interfaces DB* shall be added in the *Interfaces DB* and be marked with the tag *new*. Slave interfaces of devices marked with *upward* shall also be marked with *upward*.

5. **Get new slaves on new found master interfaces**

All master interfaces marked with *new* in the *Interfaces DB* shall be sent a *Slaves Request*. The tag *new* shall then be removed from the scanned interfaces in the *Interfaces DB*. All found slave devices that are not yet present in the *Devices DB* shall be added into it and mark with the tag *new*.

6. **Get new masters on new found slave interfaces**

All slave interfaces marked with *new* and *upward* in the *Interfaces DB* shall be sent a *BrowseOnSlaveInterface Request*. The tags *new* and *upward* shall then be removed from the scanned slave interfaces in the *Interfaces DB*. All found master devices that are not yet present in the *Devices DB* shall be added into it and mark with the tag *new* and *upward*.

7. **Check for not handled devices**

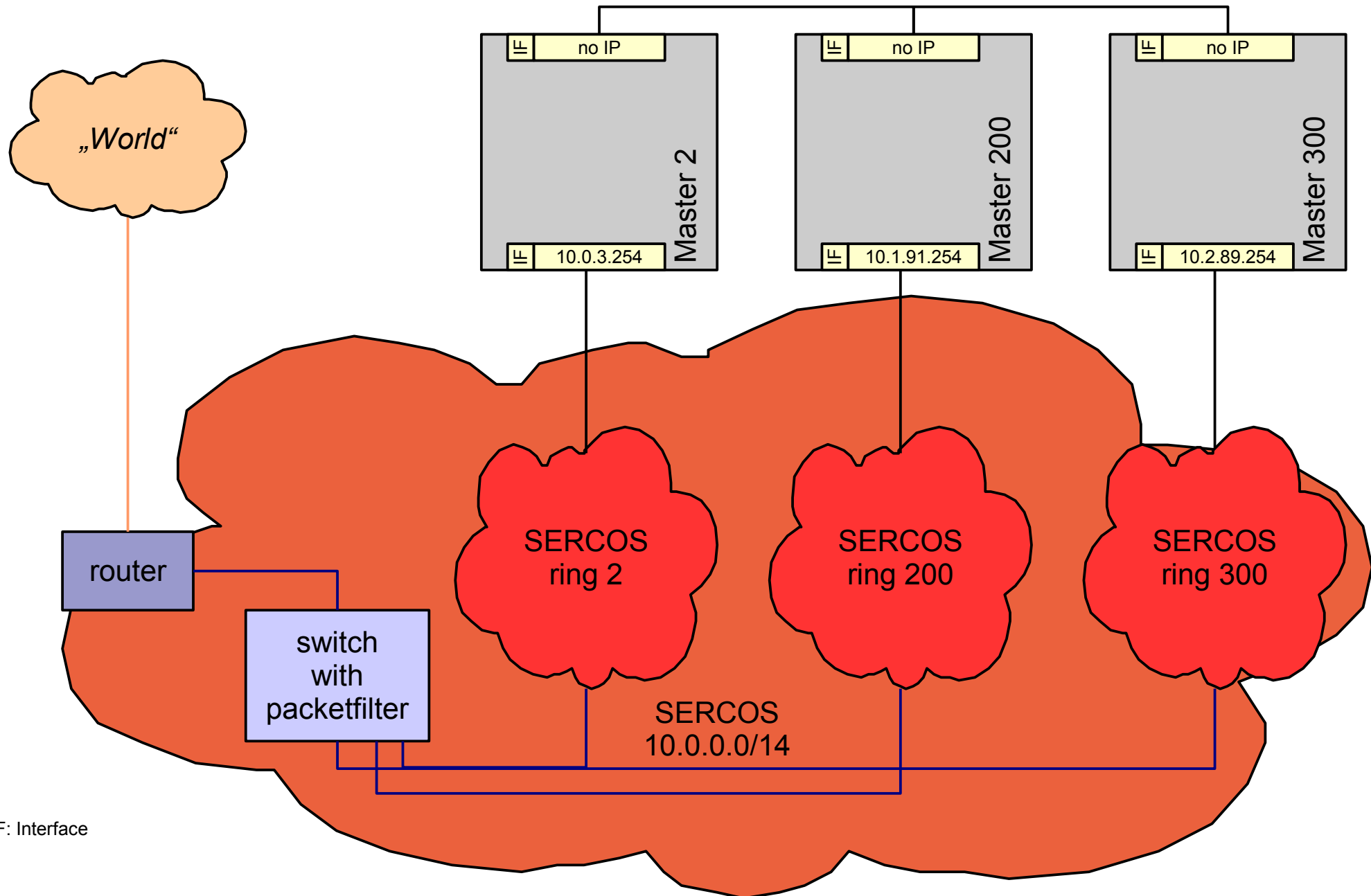
If devices are marked with the tag *new* in the *Devices DB*, proceed with step 4.

8. **Result**

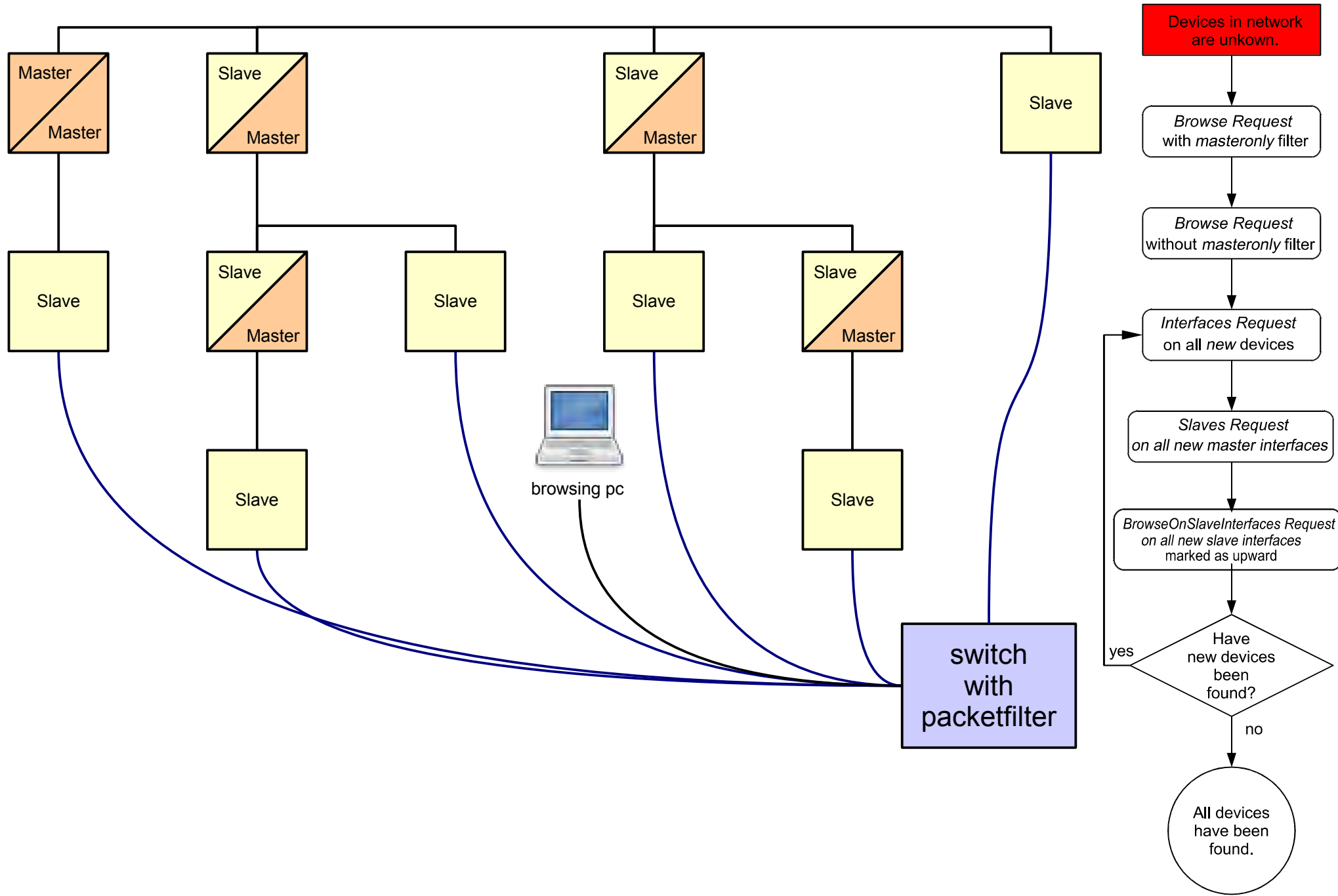
All devices and information of its type and interfaces have been found.

Reference: A detailed example of remote browsing.

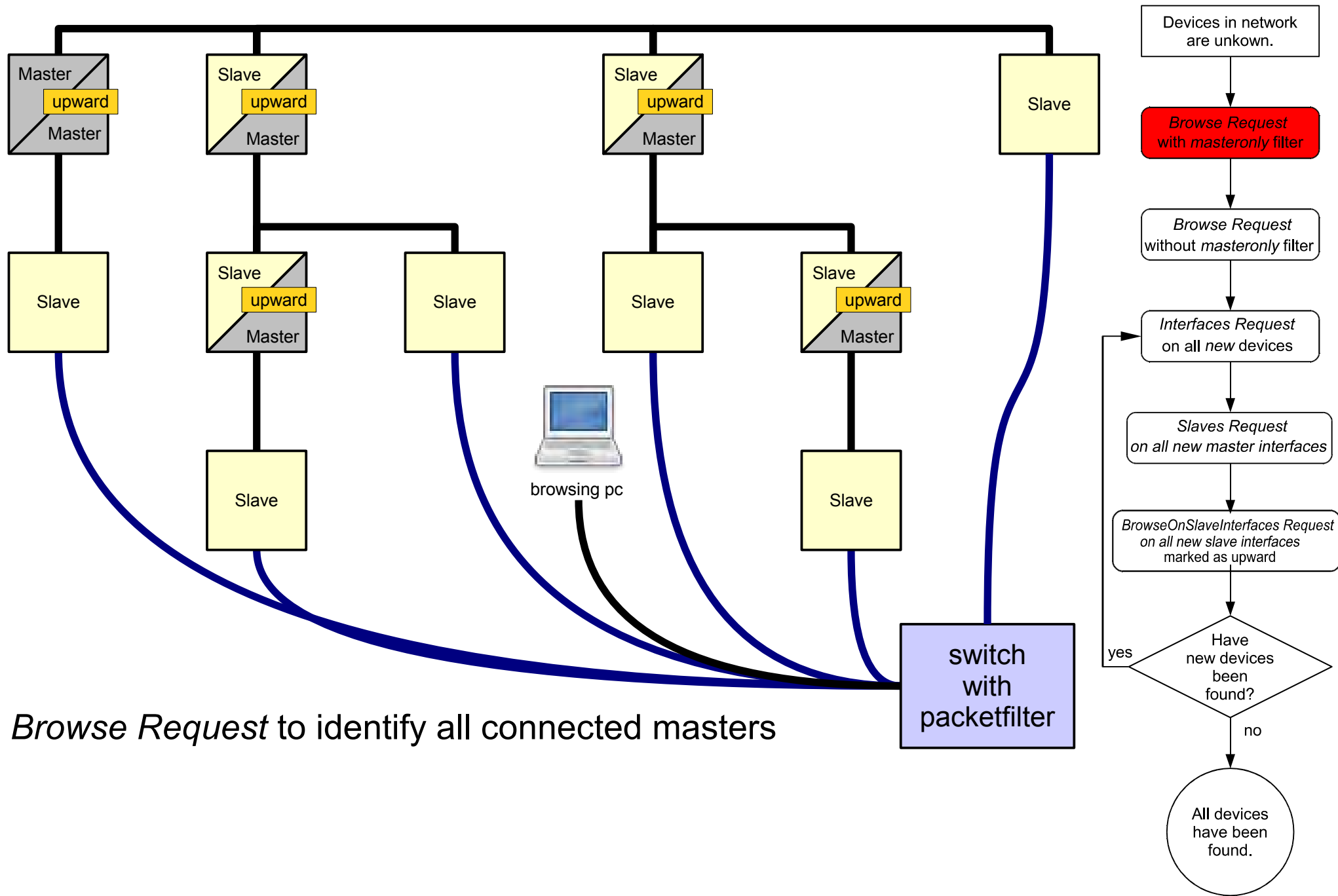
Broadcast Topology



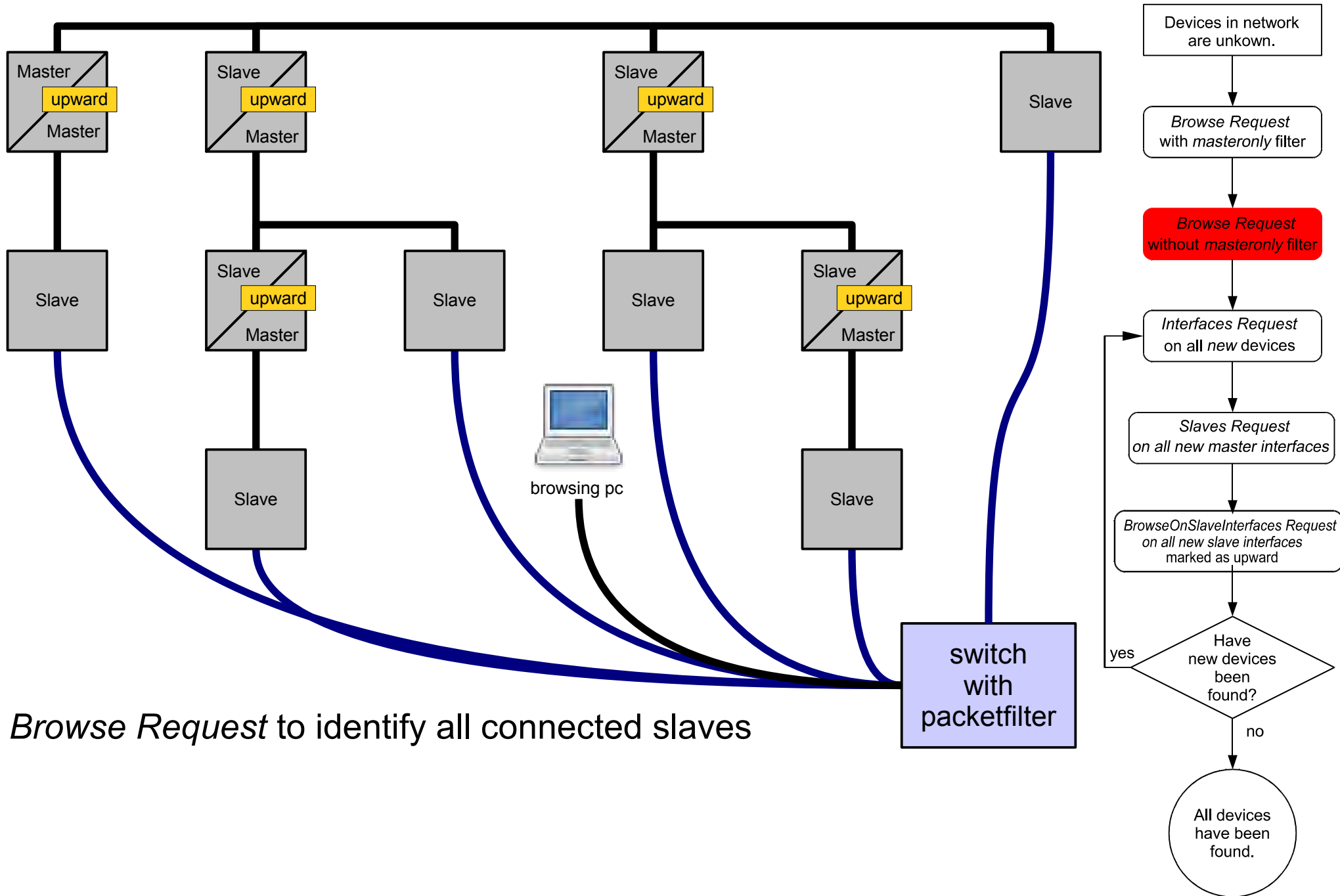
Example for Remote Browsing Broadcast Topology



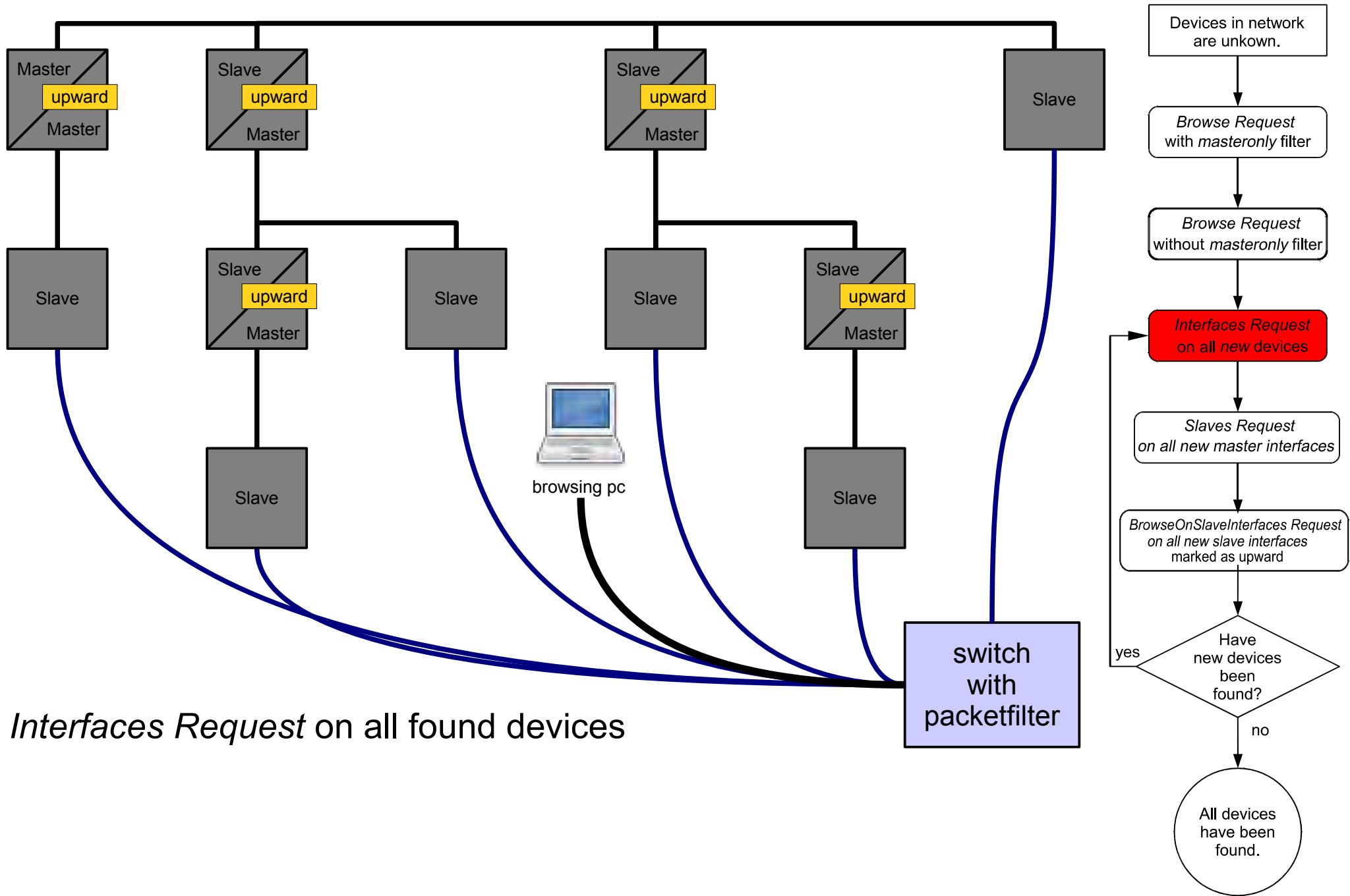
Browsing local network for masters



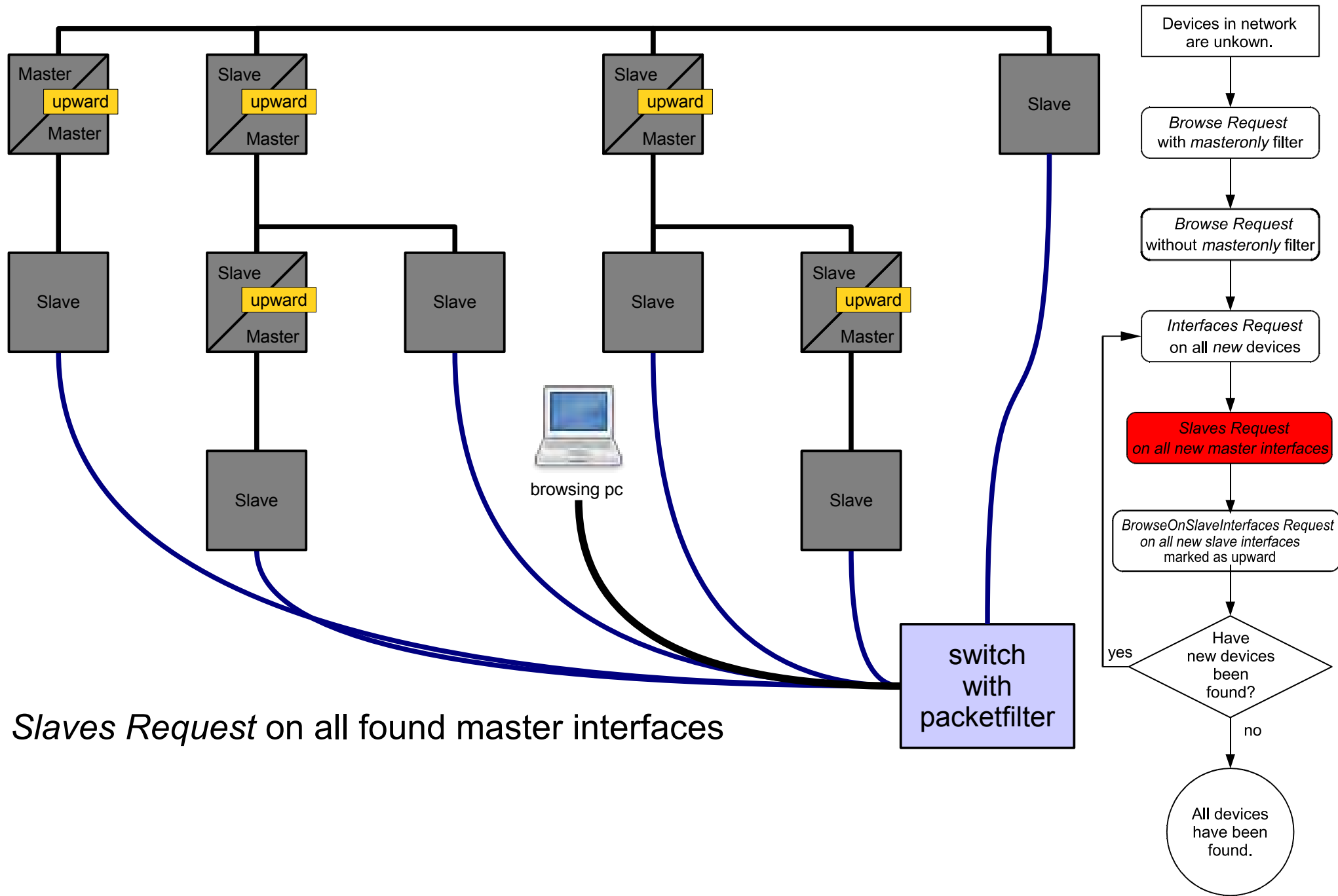
Browsing local network for slaves



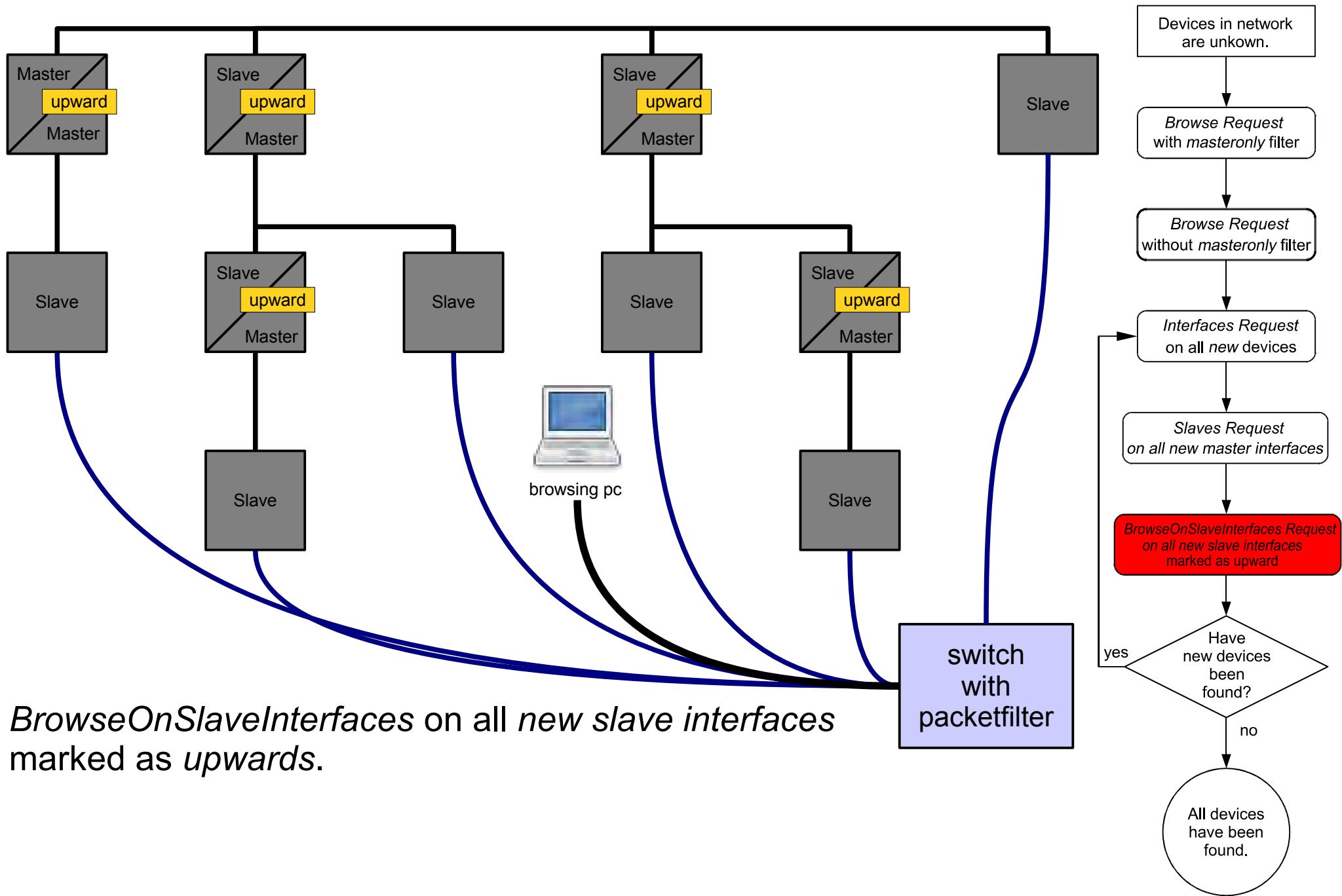
Interfaces Request



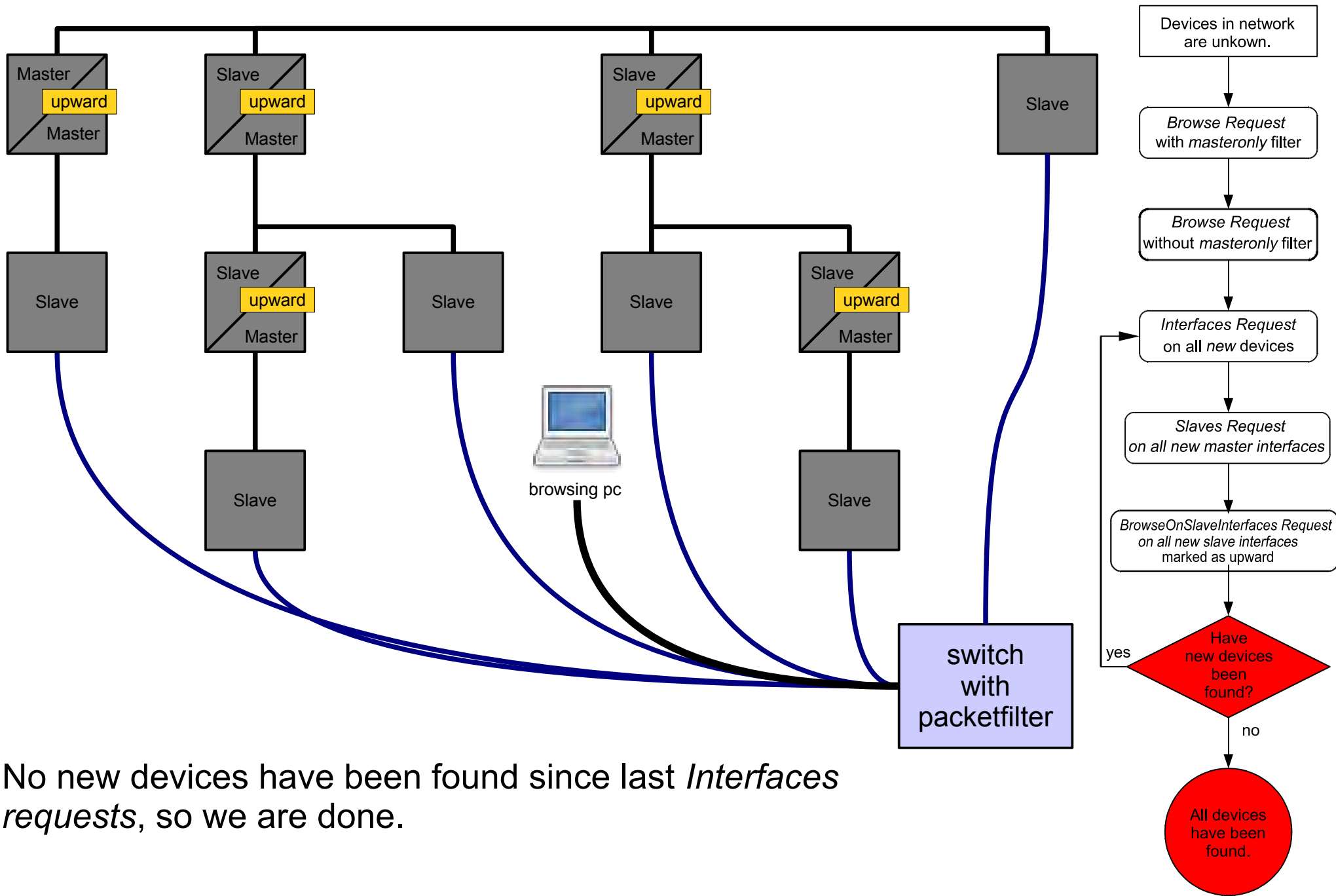
Slaves Request



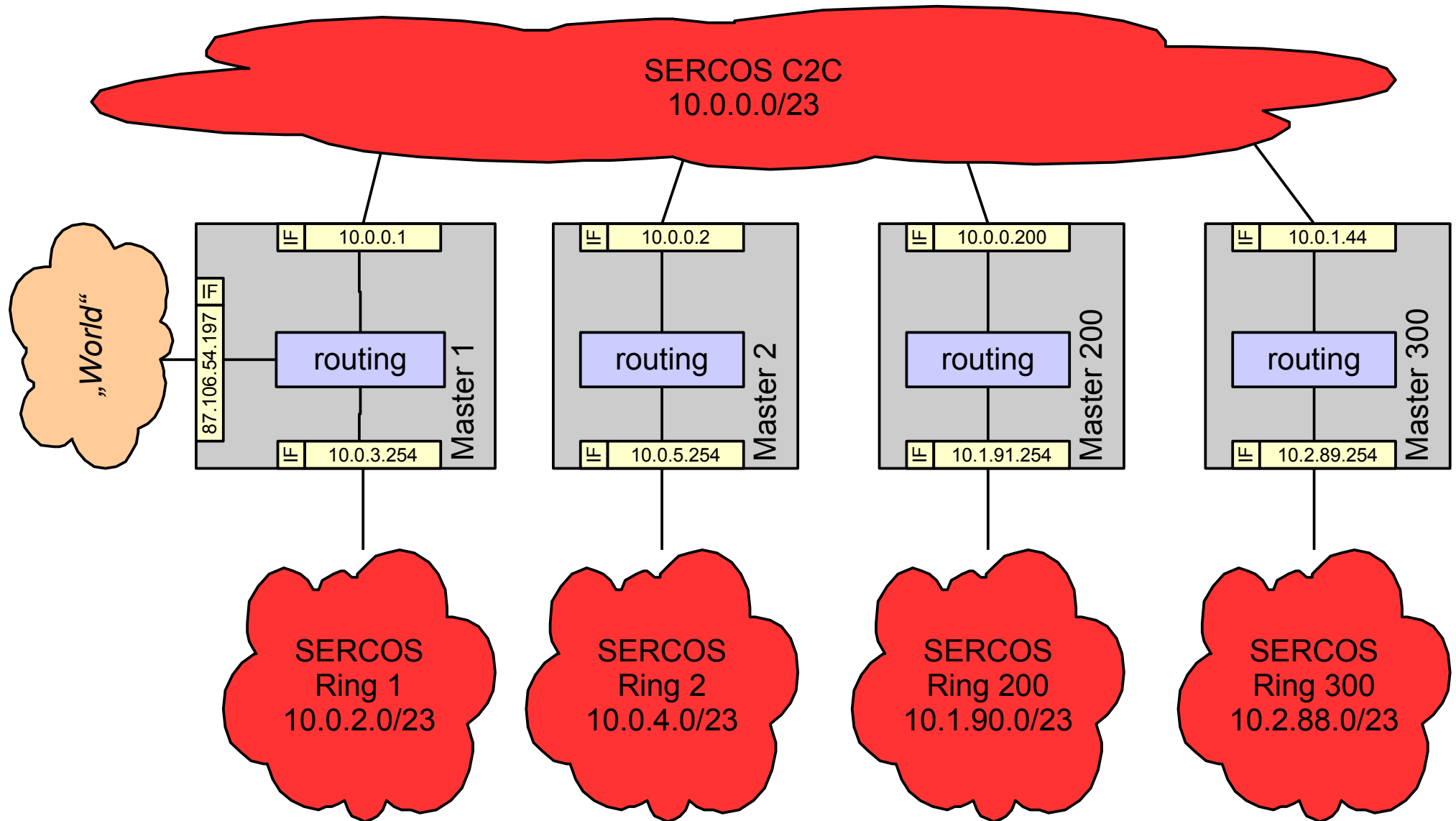
Browse on new slave interfaces of upward devices



New devices found?

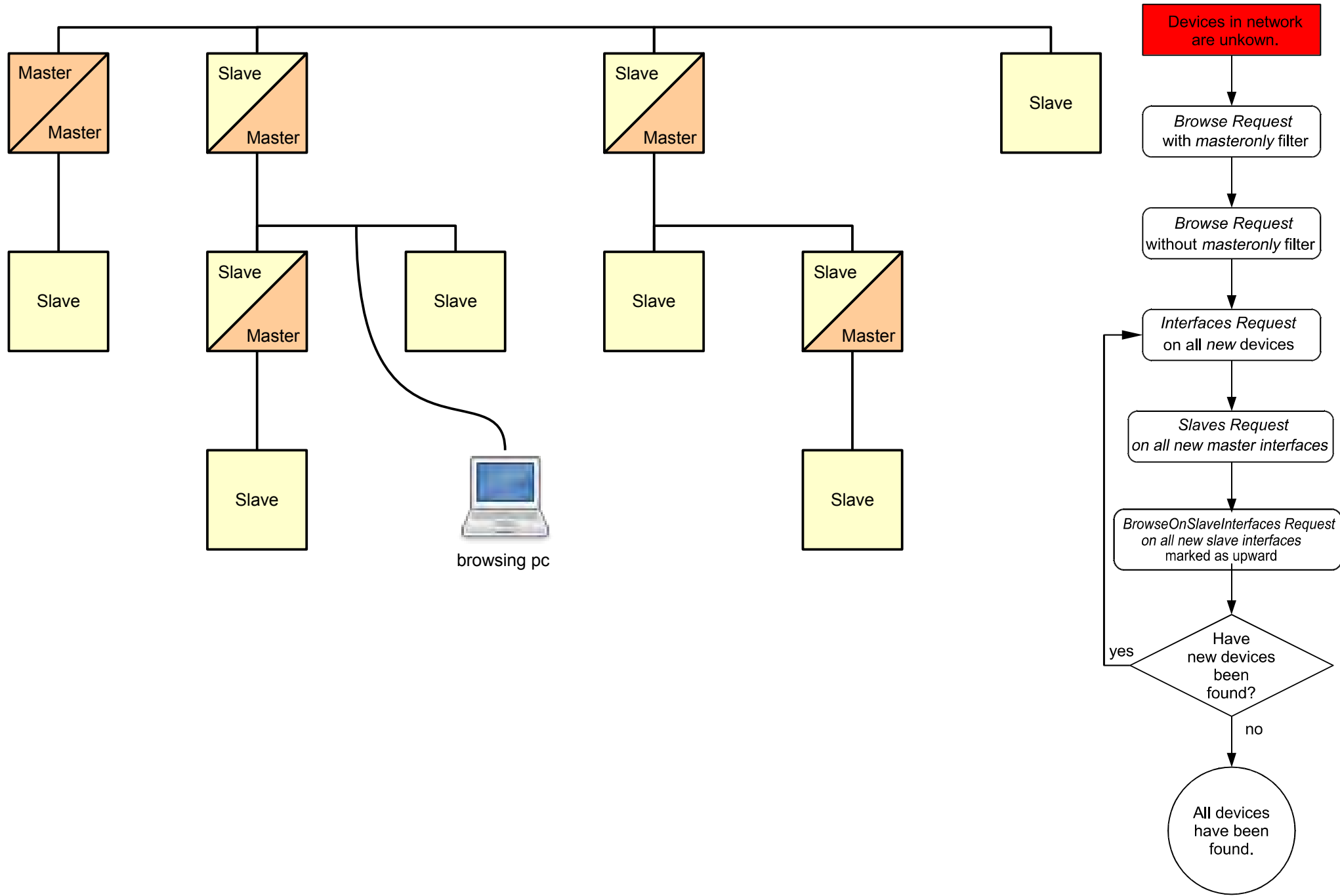


Routing Topology

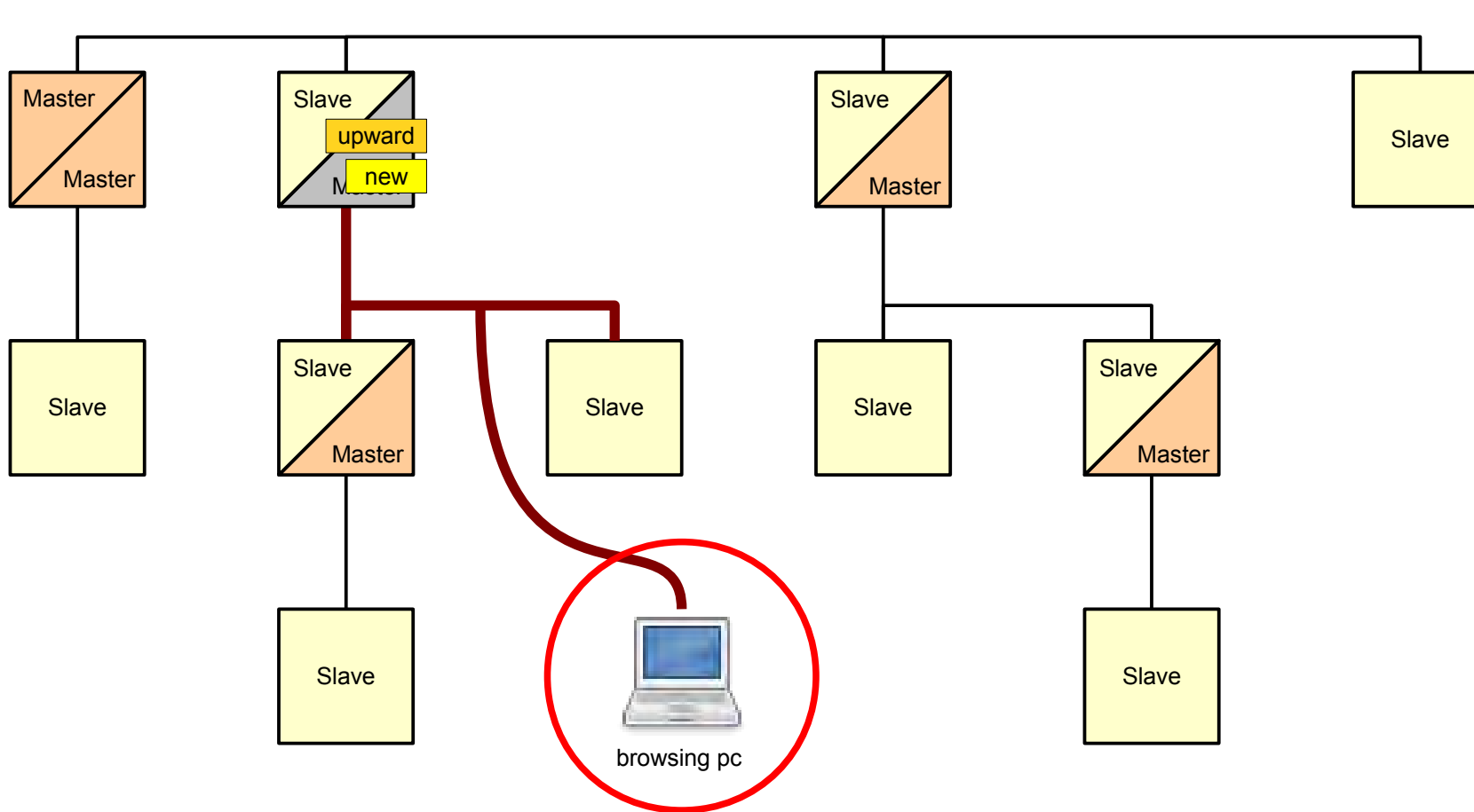


IF: Interface NAT: Native Address Translation

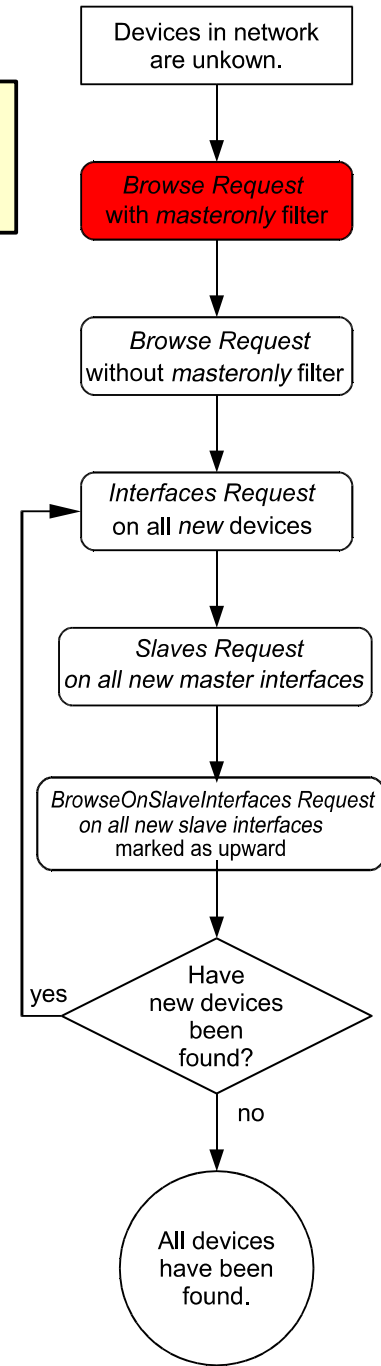
Example for Remote Browsing with Routing Topology



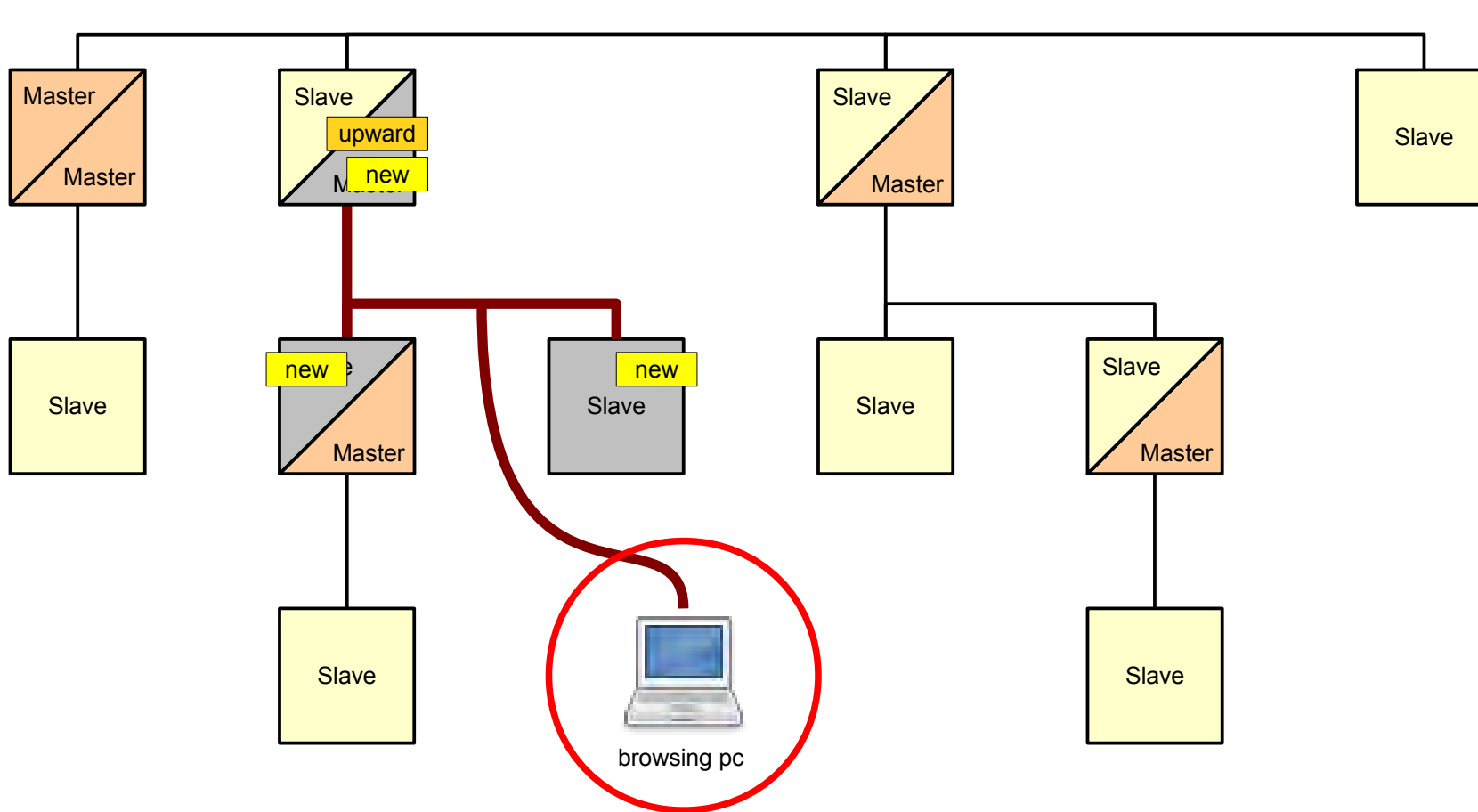
Browsing local network for masters



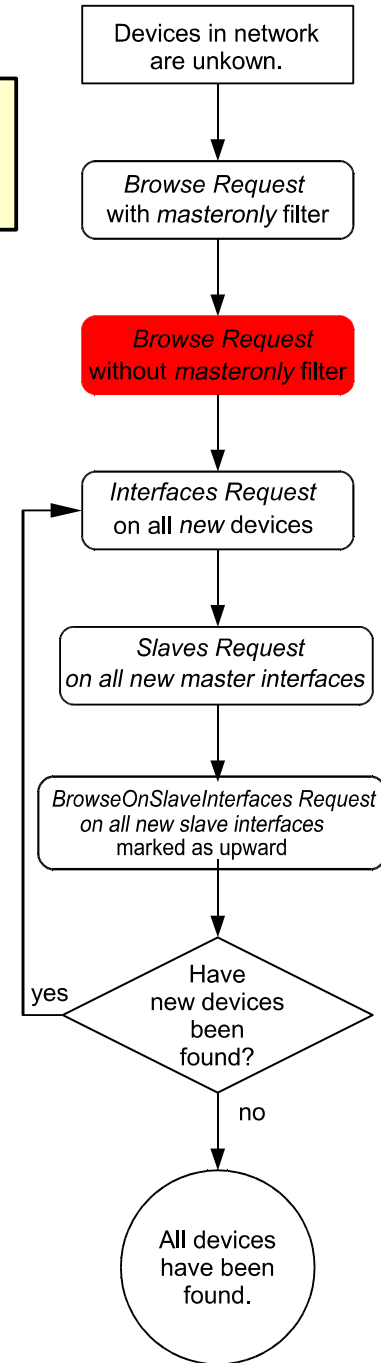
Browse Request to identify all connected masters



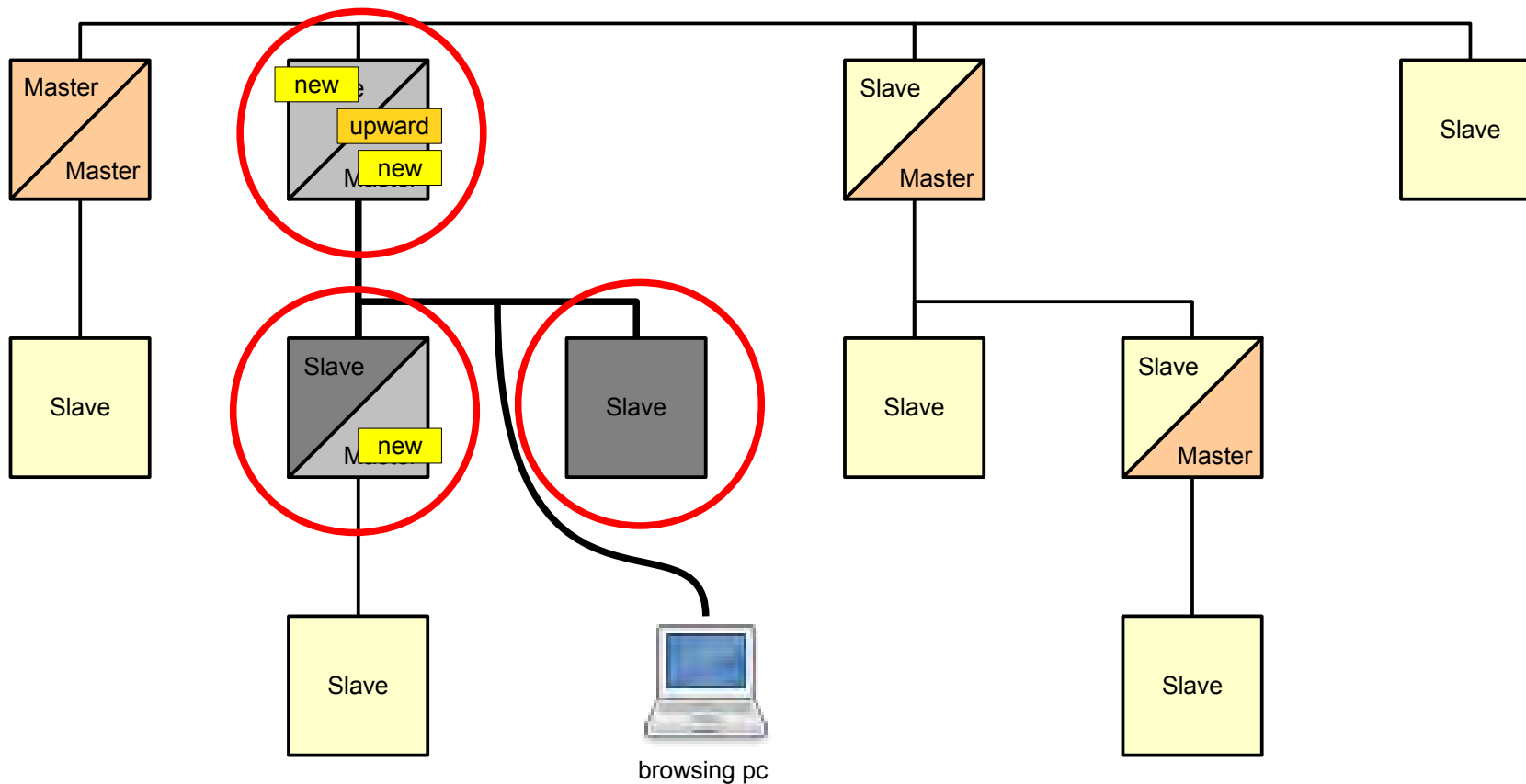
Browsing local network for slaves



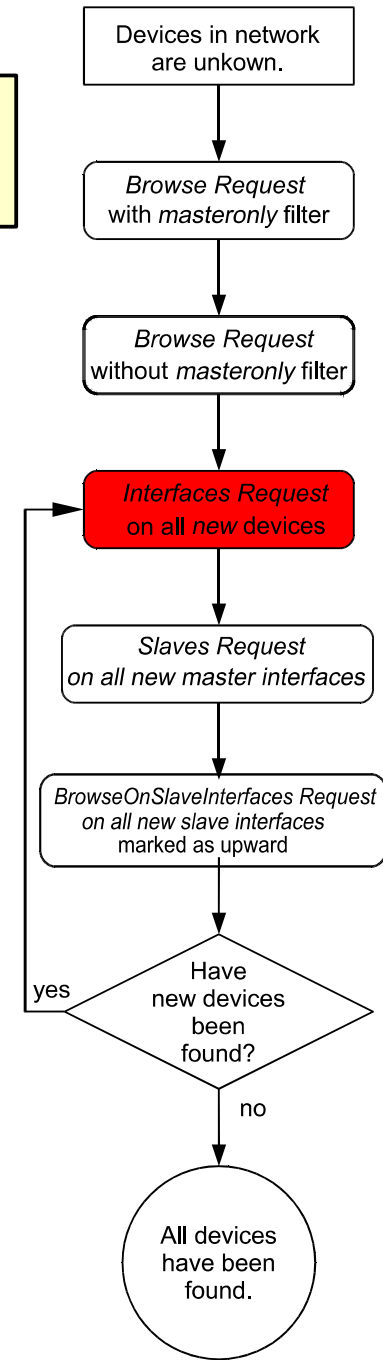
Browse Request to identify all connected slaves



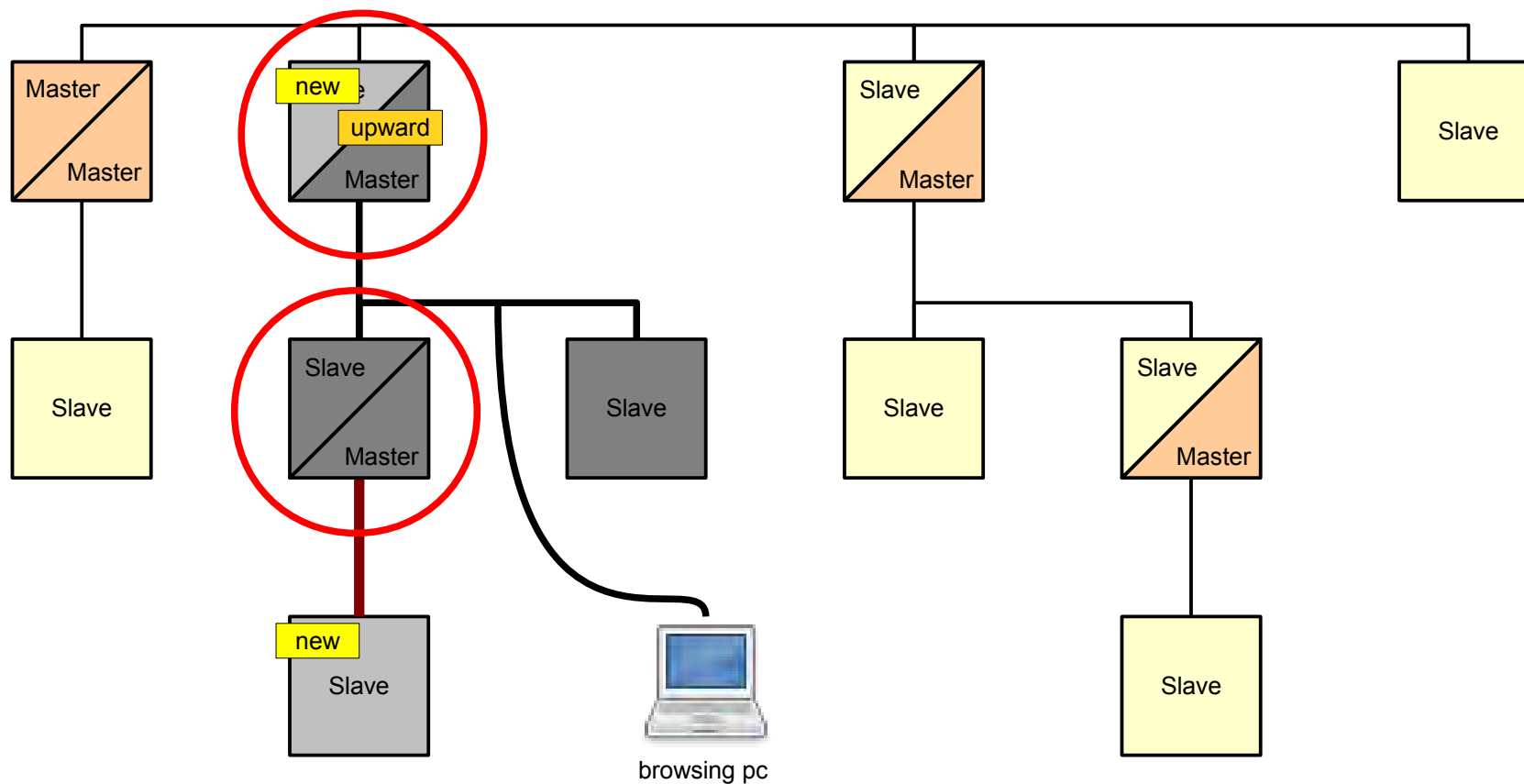
Interfaces Request



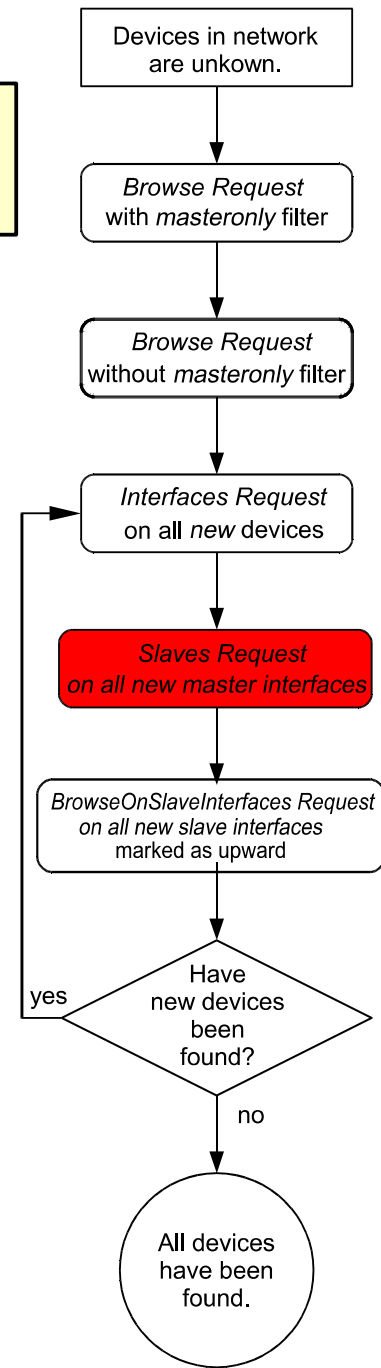
Interfaces Request on all found devices



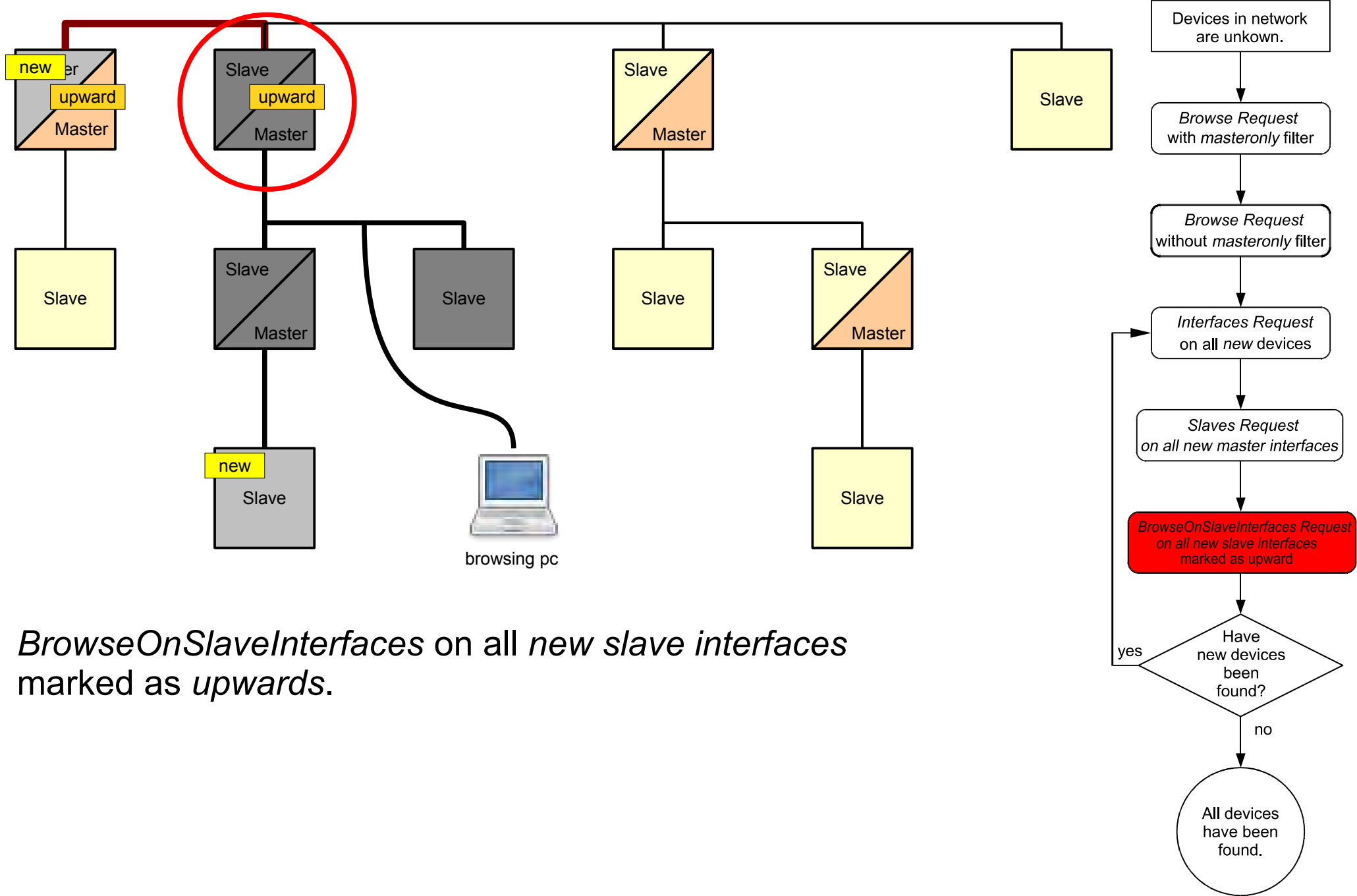
Slaves Request



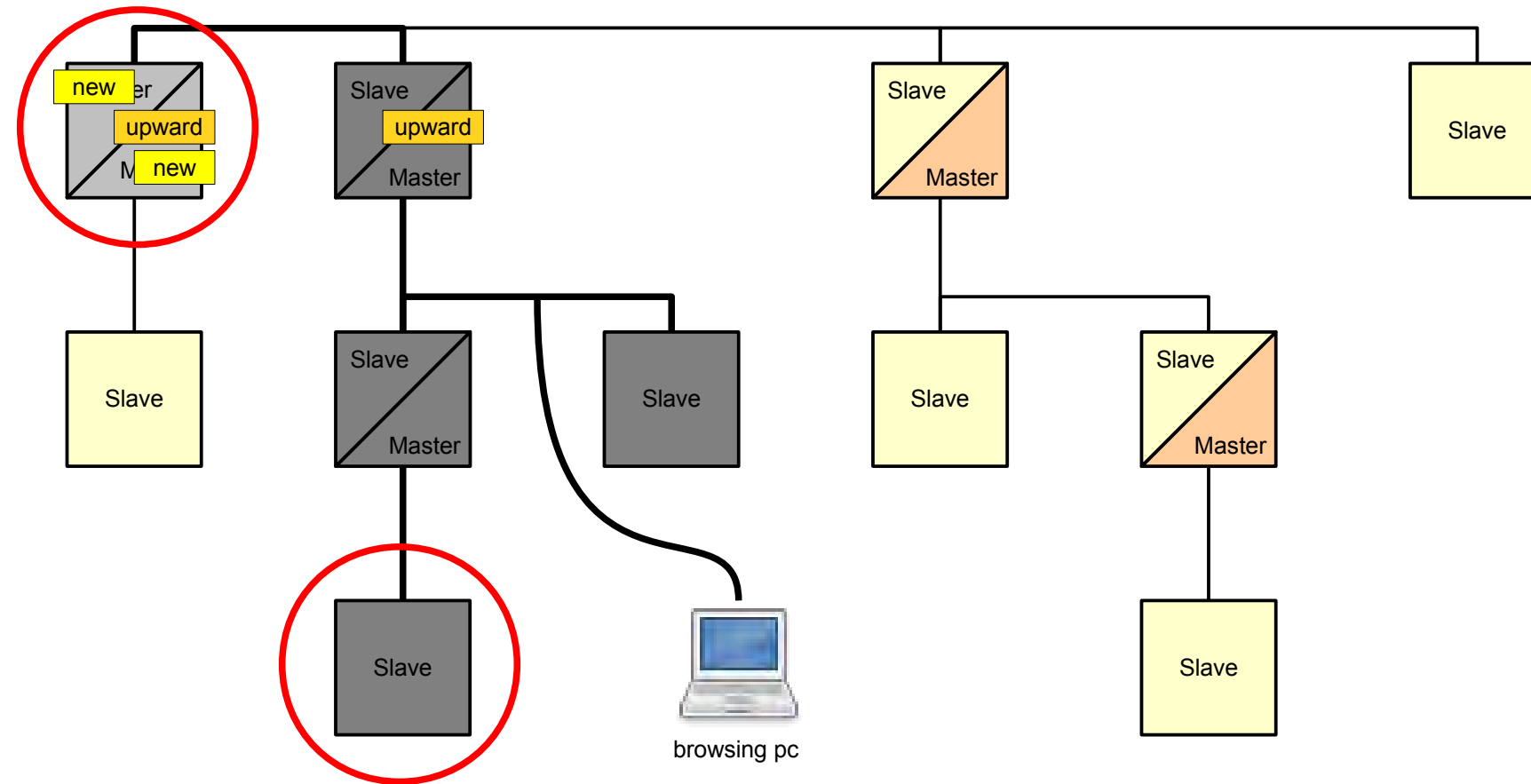
Slaves Request on all found master interfaces.



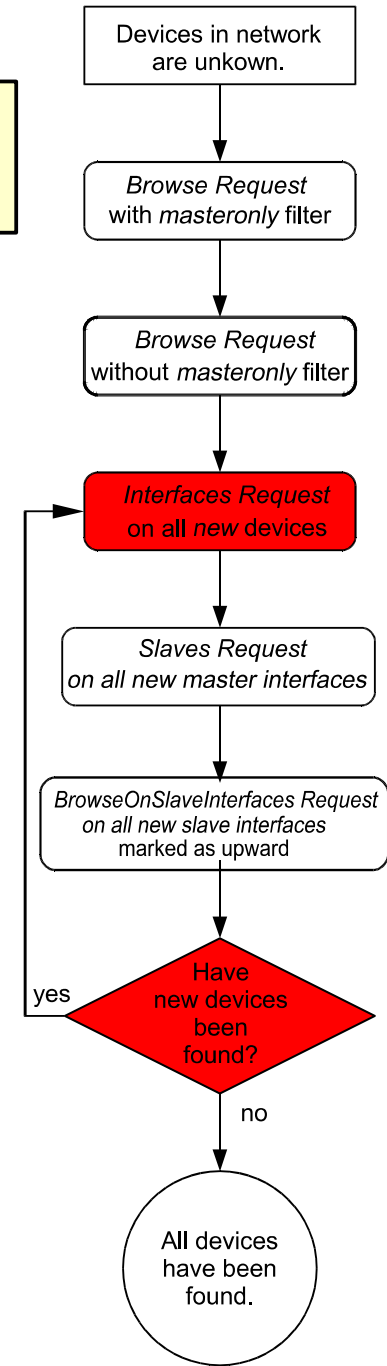
Browse on new slave interfaces of upward devices



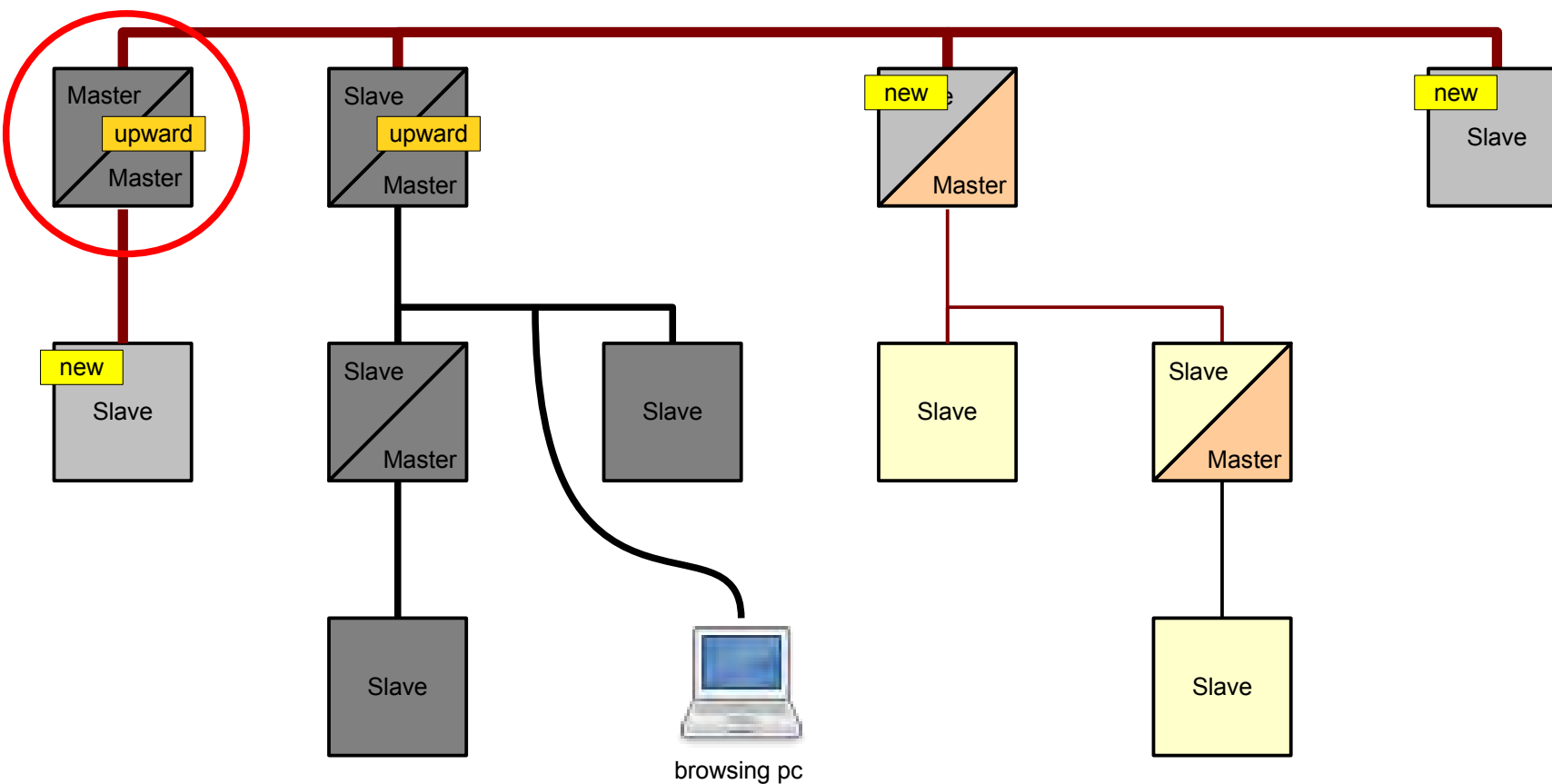
Interfaces Request on all new found interfaces



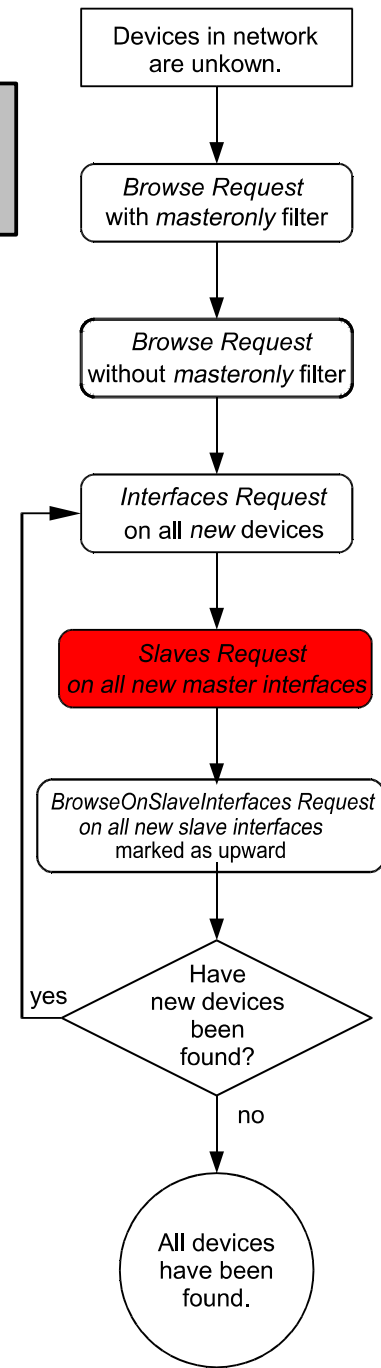
Interfaces Request on all new found devices



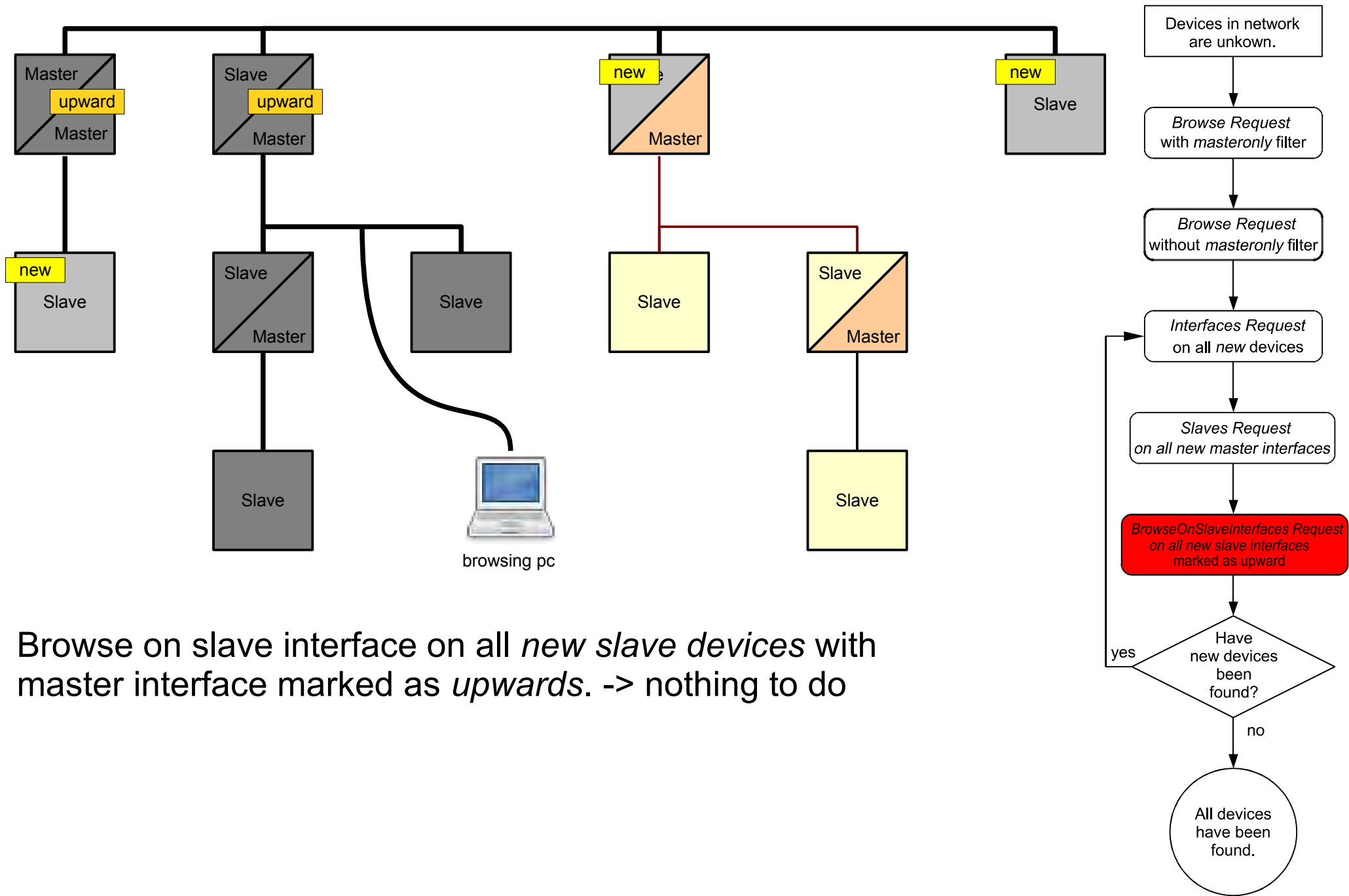
Slaves Request



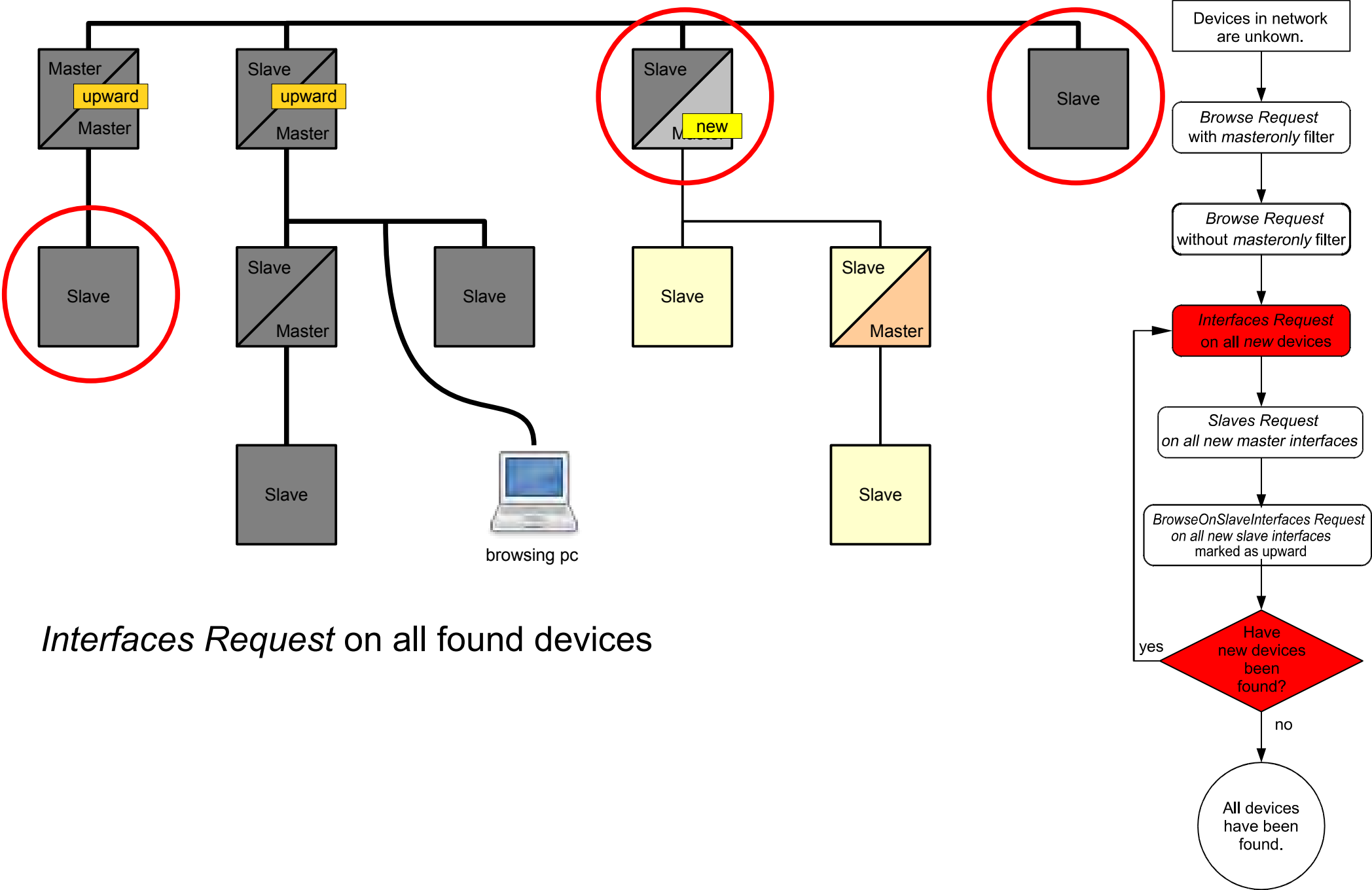
Slaves Request on all found master interfaces



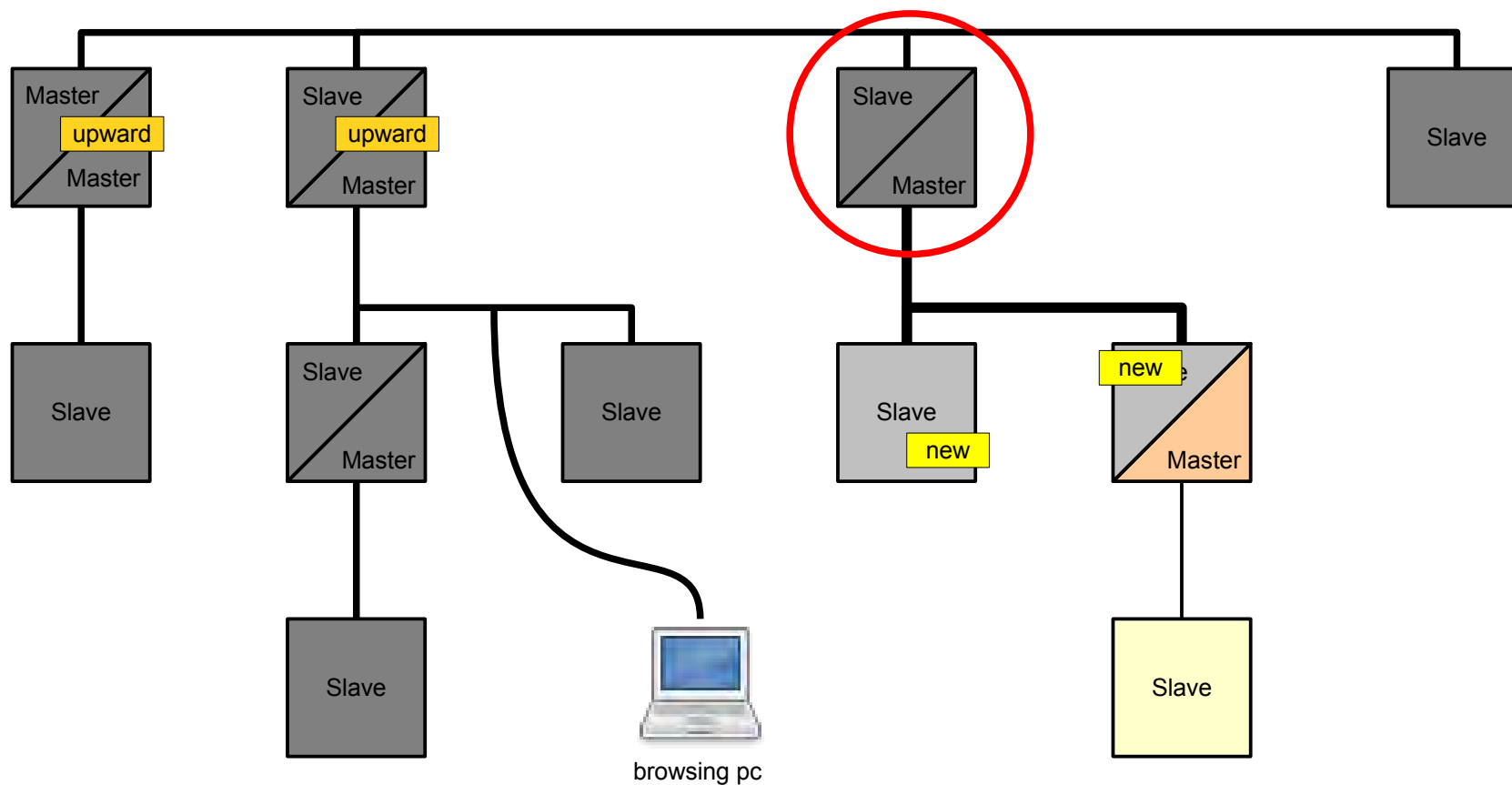
Browse on new slave interfaces of upward devices



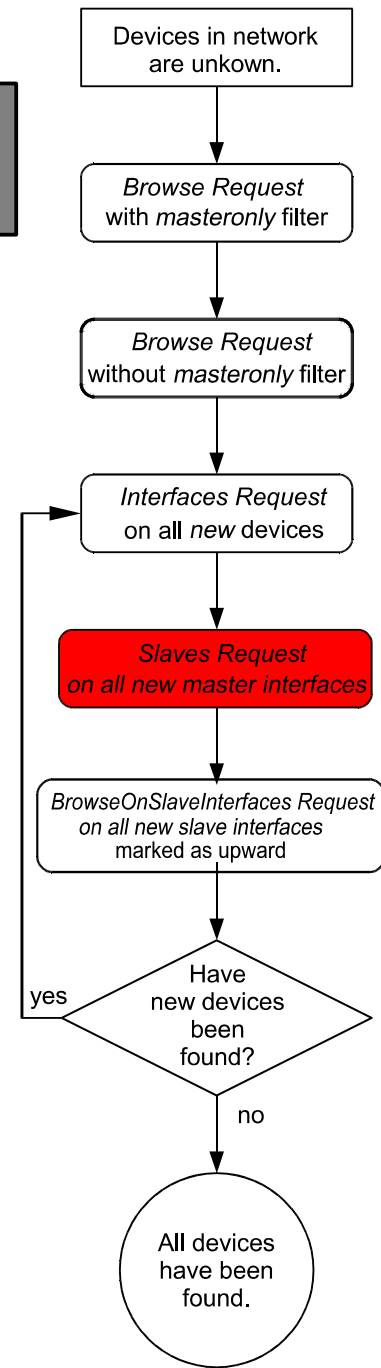
Interfaces Request



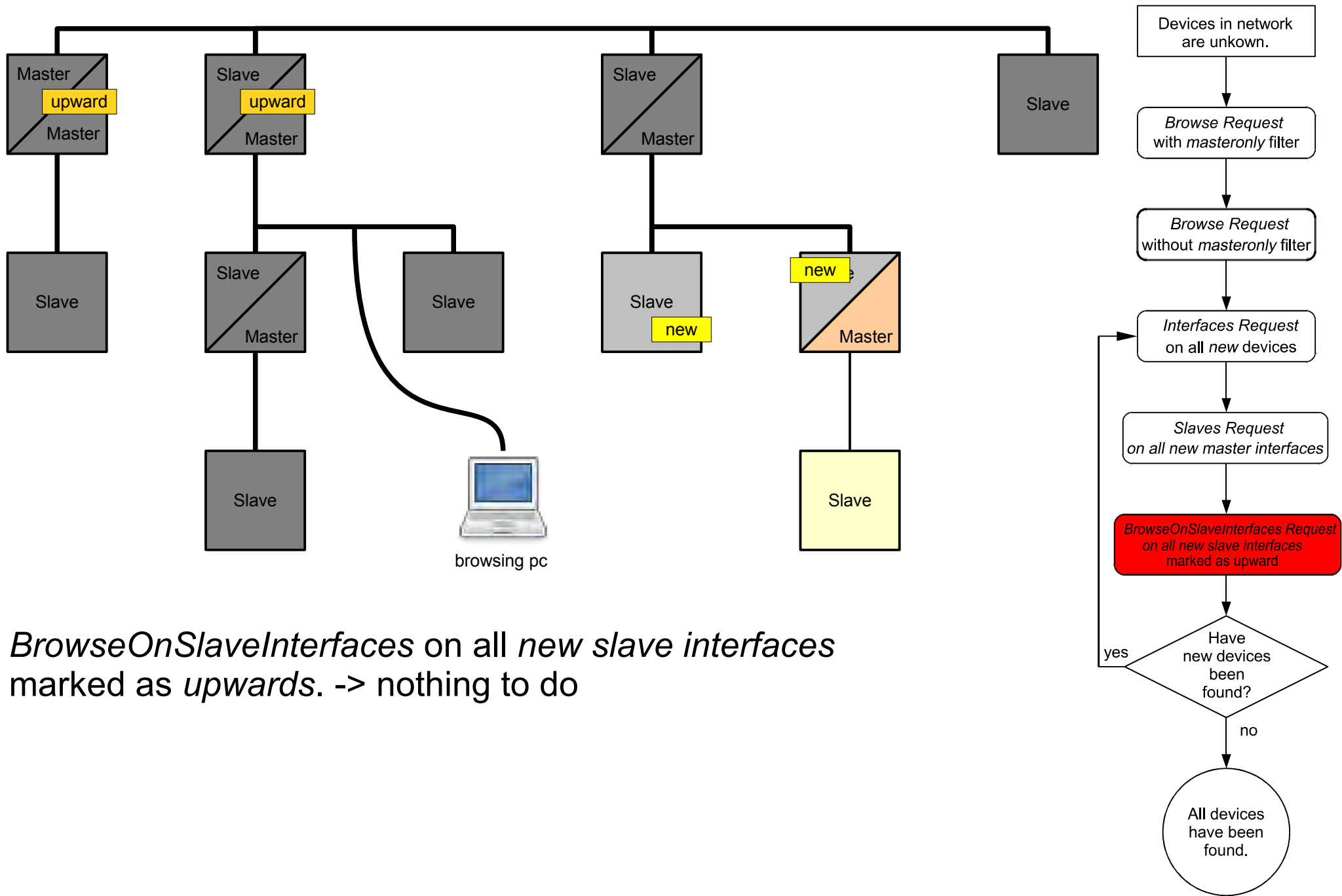
Slaves Request



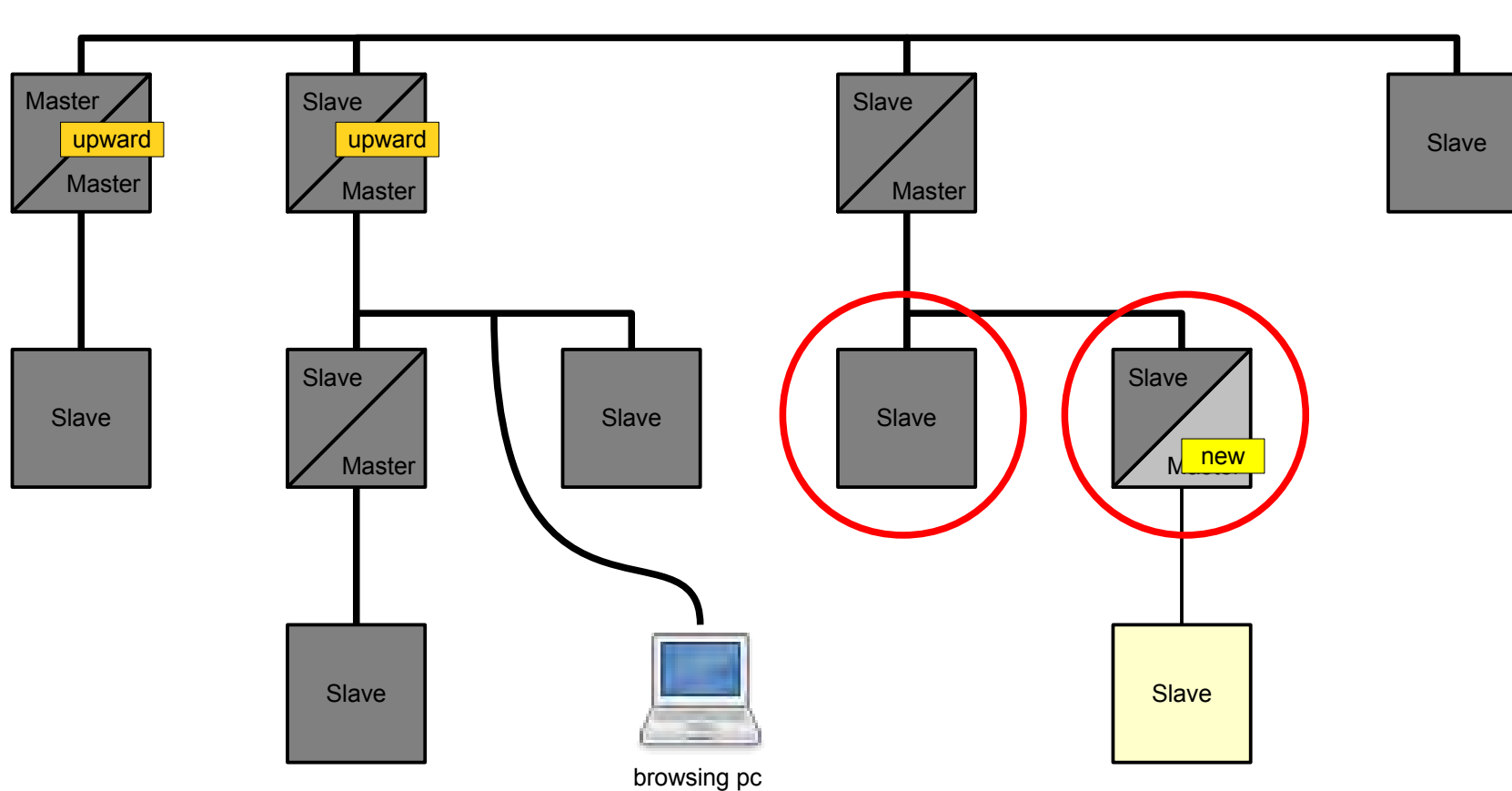
Slaves Request on all found master interfaces



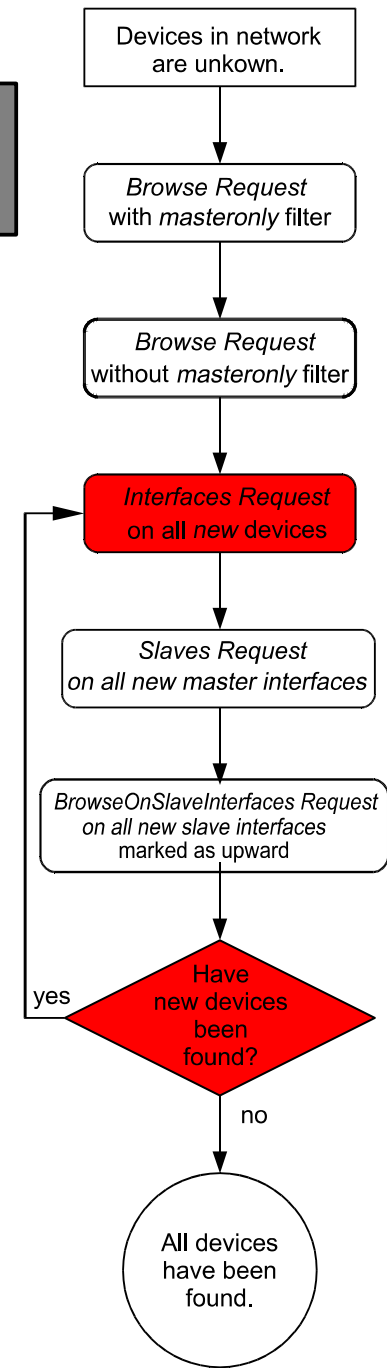
Browse on new slave interfaces of upward devices



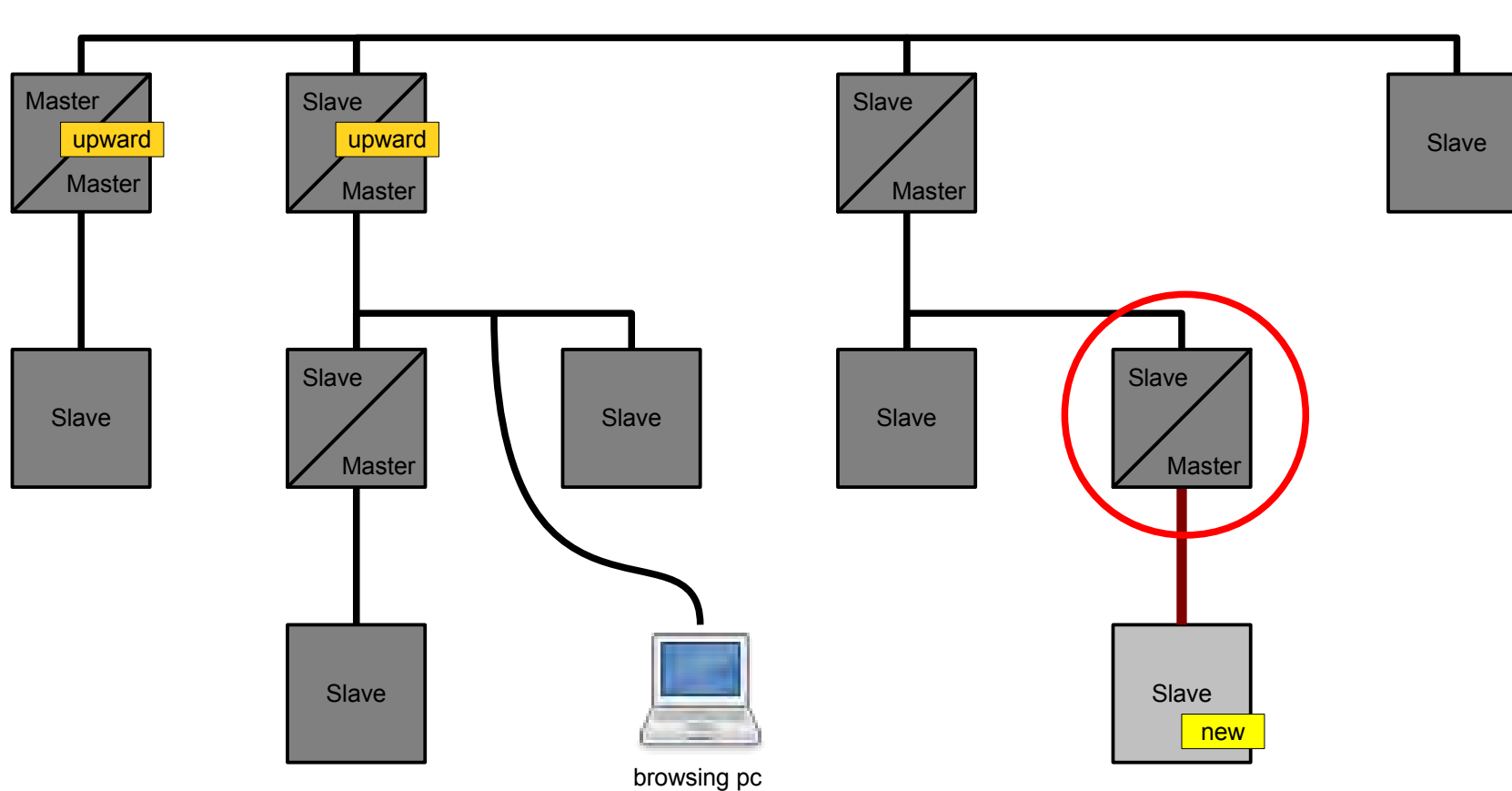
New devices found?



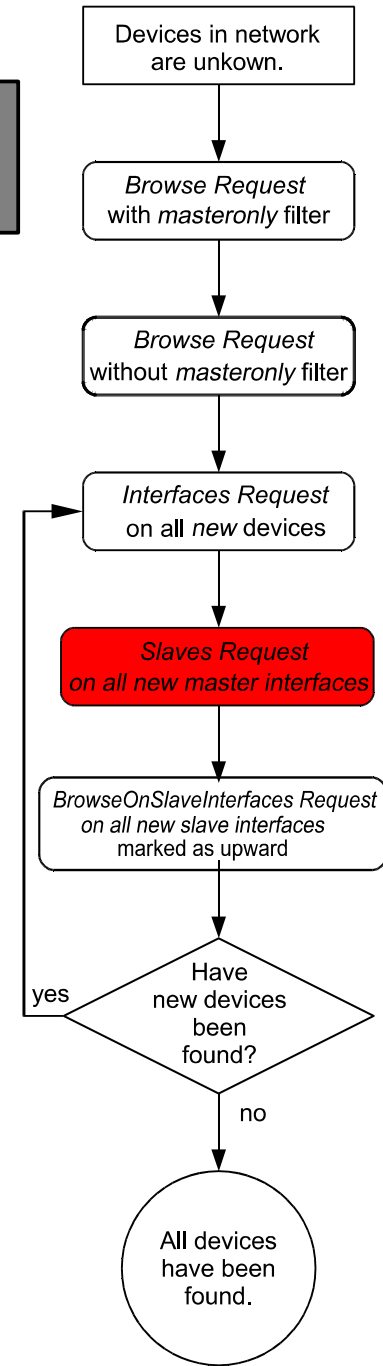
Interfaces Request on all found devices



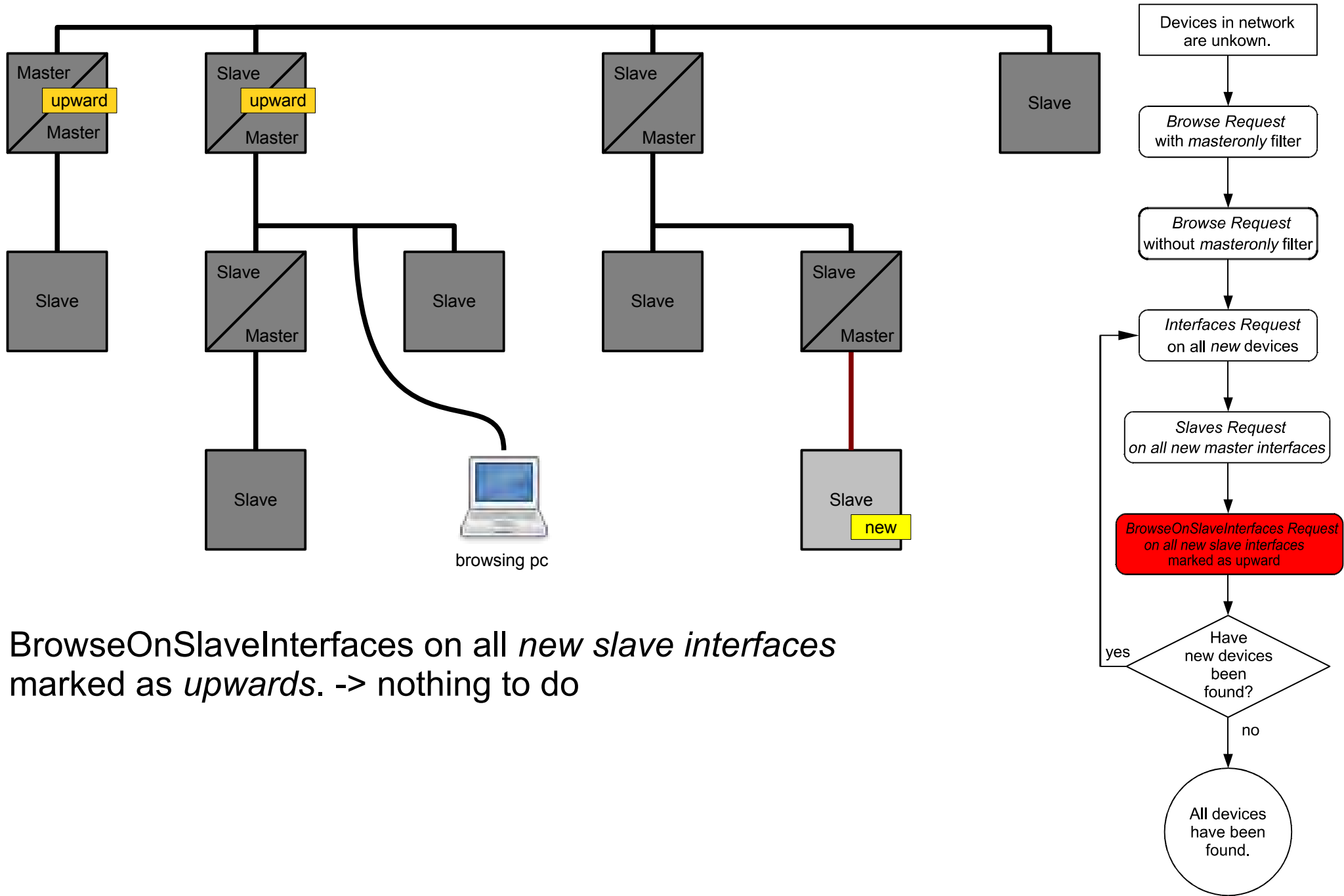
Slaves Request



Slaves Request on all found master interfaces

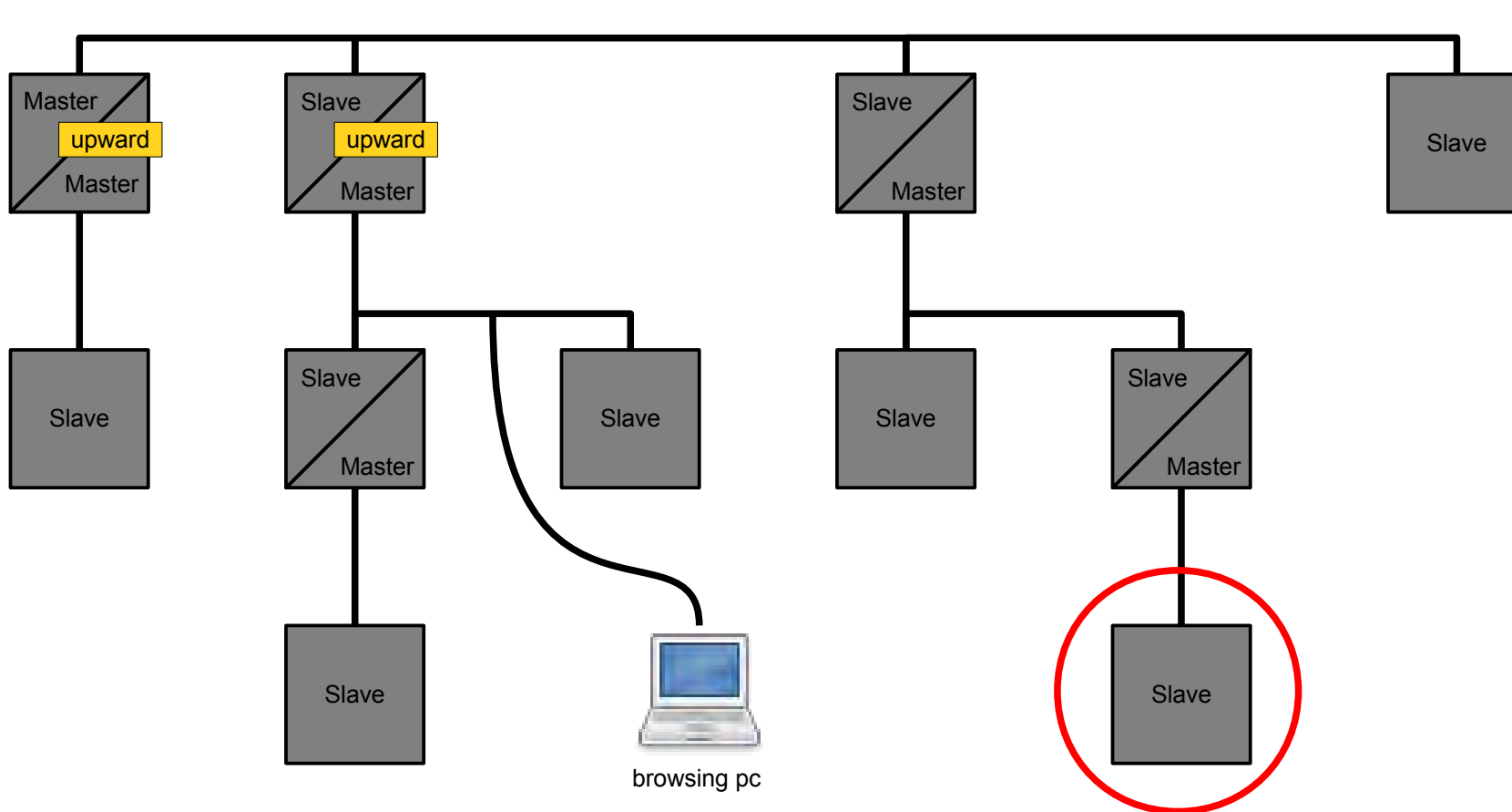


Browse on new slave interfaces of upward devices



BrowseOnSlaveInterfaces on all *new slave interfaces* marked as *upwards*. -> nothing to do

New devices found?



Interfaces Request on all found devices

